

第1章

初识Java

( 视频讲解：34分钟)

Java是一种跨平台的、面向对象的程序设计语言。本章将简单介绍Java语言的不同版本及其相关特性以及学好Java语言的方法等，主要目的是让读者对Java语言有一个整体的了解，然后再慢慢地学习具体内容，最后达到完全掌握Java语言的目的。

通过阅读本章，您可以：

- ▶▶ 了解Java语言
- ▶▶ 了解Java的版本
- ▶▶ 了解Java的应用领域
- ▶▶ 了解如何学好Java
- ▶▶ 了解Java语言特性
- ▶▶ 掌握不同平台的JDK环境搭建
- ▶▶ 掌握Java程序的编写方法

1.1 Java简介

Java是一种高级的面向对象的程序设计语言。使用Java语言编写的程序是跨平台的，从PC机到手持电话都有Java开发的程序和游戏，Java程序可以在任何计算机、操作系统和支持Java的硬件设备上运行。

1.1.1 什么是Java语言



视频讲解：光盘\TM\lx\1\什么是Java语言.exe

Java是于1995年由Sun公司推出的一种极富创造力的面向对象的程序设计语言，它是由有Java之父之称的Sun研究院院士詹姆斯·戈士林博士亲手设计而成的，并完成了Java技术的原始编译器和虚拟机。Java最初的名字是OAK，在1995年被重命名为Java，正式发布。

Java是一种通过解释方式来执行的语言，其语法规则和C++类似。同时，Java也是一种跨平台的程序设计语言。用Java语言编写的程序，可以运行在任何平台和设备上，如跨越IBM个人电脑、MAC苹果计算机、各种微处理器硬件平台，以及Windows、UNIX、OS/2、MAC OS等系统平台，真正实现“一次编写，到处运行”。Java非常适于企业网络和Internet环境，并且已成为Internet中最具有影响力、最受欢迎的编程语言之一。

与目前常用的C++相比，Java语言简洁得多，而且提高了可靠性，除去了最大的程序错误根源，此外它还有较高的安全性，可以说它是有史以来最为卓越的编程语言。

Java语言编写的程序既是编译型的，又是解释型的。程序代码经过编译之后转换为一种称为Java字节码的中间语言，Java虚拟机（JVM）将对字节码进行解释和运行。编译只进行一次，而解释在每次运行程序时都会进行。编译后的字节码采用一种针对JVM优化过的机器码形式保存，虚拟机将字节码解释为机器码，然后在计算机上运行。Java语言程序代码的编译和运行过程如图1.1所示。

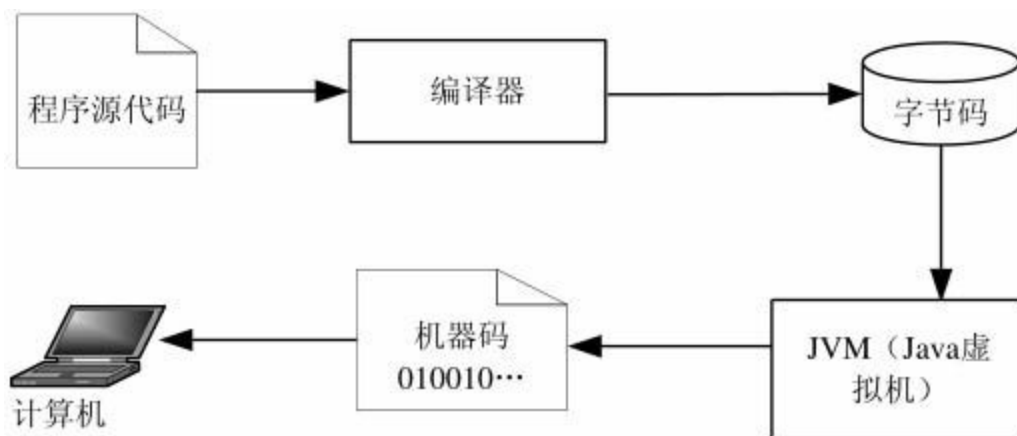


图1.1 Java程序的编译和运行过程

1.1.2 Java的应用领域



视频讲解：光盘\TM\lx\1\Java的应用领域.exe

借助Java，程序开发人员可以自由地使用现有的硬件和软件系统平台。这是因为Java是独立于平台的，它还可以应用于计算机之外的领域，Java程序可以在便携式计算机、电视、电话、手机和其他的大量电子设备上运行。Java的用途不胜枚举，它拥有无可比拟的能力，其节省的时间和费用也十分可观。Java的应用领域主要有以下方面：

- ☒ 桌面应用系统开发。
- ☒ 嵌入式系统开发。
- ☒ 电子商务应用。
- ☒ 企业级应用开发。
- ☒ 交互式系统开发。
- ☒ 多媒体系统开发。
- ☒ 分布式系统开发。
- ☒ Web应用系统开发。

Java无处不在，它已经拥有几百万个用户，其发展速度要快于在它之前的任何一种计算机语言。Java能够给企业和最终用户带来数不尽的好处。Oracle公司董事长和首席执行官Larry Ellison说过：“Java正在进入企业、家庭和学校。它正在像Internet本身一样成为普遍存在的技术。”

如果仔细观察，就会发现Java就在我们身边，如我们经常使用的Java开发工具Eclipse、NetBeans、JBuilder等，另外还有RSSOwl、Limewire、Azureus、CyberDuck、OpenOffice等优秀软件都是使用Java编写的。此外，各手机厂商都为自己的产品提供了Java技术的支持，各种手机上的Java程序和游戏已经数不胜数。

1.1.3 Java的版本



视频讲解：光盘\TM\lx\1\Java的版本.exe

自从Sun公司推出Java以来，就力图使之无所不能。Java发展至今，按应用范围分为3个版本，即Java SE、Java EE和Java ME，也就是Sun ONE（Open Net Environment）体系。本节将分别介绍这3个Java版本。

1. Java SE

Java SE是Java的标准版，主要用于桌面应用程序的开发，同时也是Java的基础，它包含Java语言基础、JDBC（Java数据库连接性）操作、I/O（输入/输出）、网络通信、多线程等技术。Java SE的结构如图1.2所示。

2. Java EE

Java EE是Java的企业版，主要用于开发企业级分布式的网络程序，如电子商务网站和ERP（企业资源规划）系统，其核心为EJB（企业Java组件模型）。Java EE的结构如图1.3所示。

3. Java ME

Java ME主要应用于嵌入式系统开发，如掌上电脑、手机等移动通信电子设备，现在大部分手机厂商所生产的手机都支持Java技术。Java ME的结构如图1.4所示。

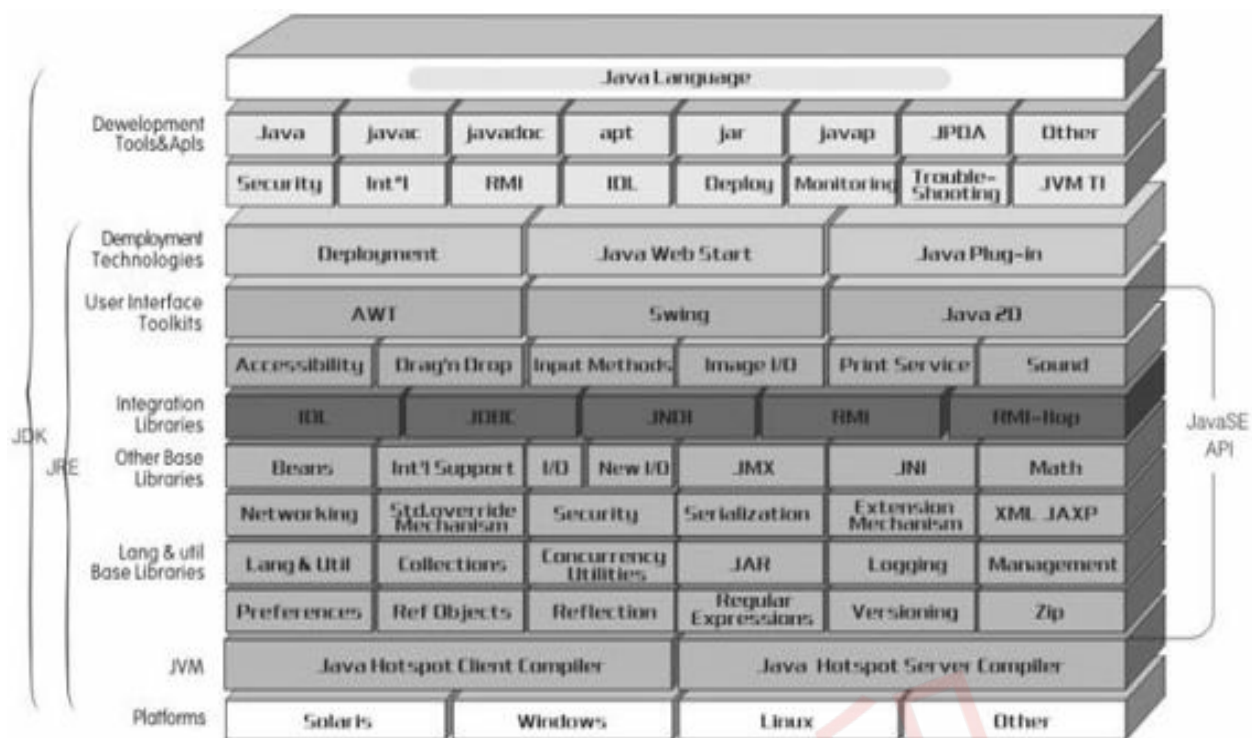


图1.2 Java SE的结构

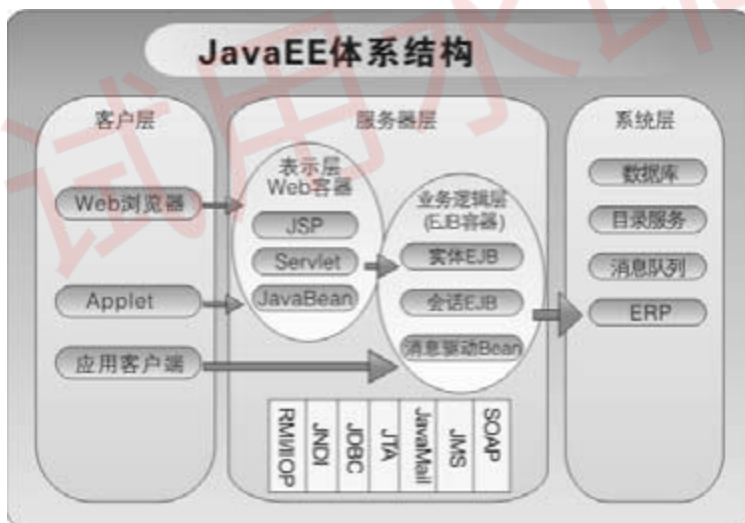


图1.3 Java EE的结构

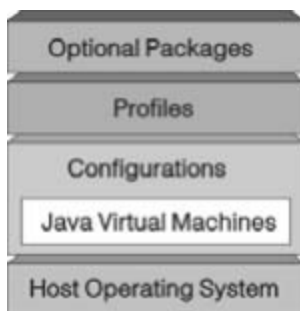


图1.4 Java ME的结构

1.1.4 怎样学好Java



视频讲解：光盘\TM\lx\1\怎样学好Java.exe

如何学好Java语言，是所有初学者需要共同面对的问题，其实每种语言的学习方法都大同小异，初学者需要注意的主要有以下几点：

- ☑ 明确自己的学习目标和大的方向，选择并锁定一门语言，按照自己的学习方向努力学习，认真研究。

- ☑ 初学者不要看太多的书，先找本相对基础的书系统地学习。很多程序开发人员工作了很久也只是熟悉部分基础而已，并没有系统地学习Java语言。

- ☑ 了解设计模式。开发程序必须编写程序代码，这些代码必须具有高度的可读性，这样编写的程序才有调试、维护和升级的价值。学习一些设计模式，能够更好地把握项目的整体结构。

- ☑ 不要死记语法。在刚接触一门语言，特别是Java语言时，掌握好基本语法，并大概了解一些功能即可。借助开发工具（如Eclipse或NetBeans）的代码辅助功能，完成代码的录入，这样可以快速进入学习状态。

- ☑ 多实践，多思考，多请教。仅读懂书本中的内容和技术是不行的，必须动手编写程序代码，并运行程序、分析运行结构，从而对学习内容有整体的认识和肯定。用自己的方式思考问题、编写代码来提高编程思想。平时多请教老师或经理，和其他人多沟通技术问题，提高自己的技术和见识。

- ☑ 不要急躁。遇到技术问题，必须冷静对待，不要让自己思维混乱，保持清醒的头脑才能分析和解决各种问题。可以尝试用听歌、散步等方式来放松自己。

- ☑ 遇到问题，首先尝试自己解决，这样可以提高自己的程序调试能力，并对常见问题有一定的了解，明白出错的原因，甚至举一反三，解决其他关联的错误问题。

- ☑ 多查阅资料。可以经常到Internet上搜索相关资料或解决问题的方法，网络上已经摘录了很多人的问题和不同的解决方法，分析这些解决问题的方法，找出最适合自己的方法。

- ☑ 多阅读别人的源代码。不但要看懂别人的程序代码，还要分析编程者的编程思想和设计模式，并融为己用。

1.1.5 Java API文档



视频讲解：光盘\TM\lx\1\Java API文档.exe

API的全称是Application Programming Interface，即应用程序编程接口。Java API文档是Java程序开发不可缺少的编程词典，它记录了Java语言中海量的API，主要包括类的继承结构、成员变量和成员方法、构造方法、静态成员的详细说明和描述信息。可以在Java网站 <http://java.sun.com> 中找到最新版本的JDK文档（即API文档），Java API文档的页面效果如图1.5所示。

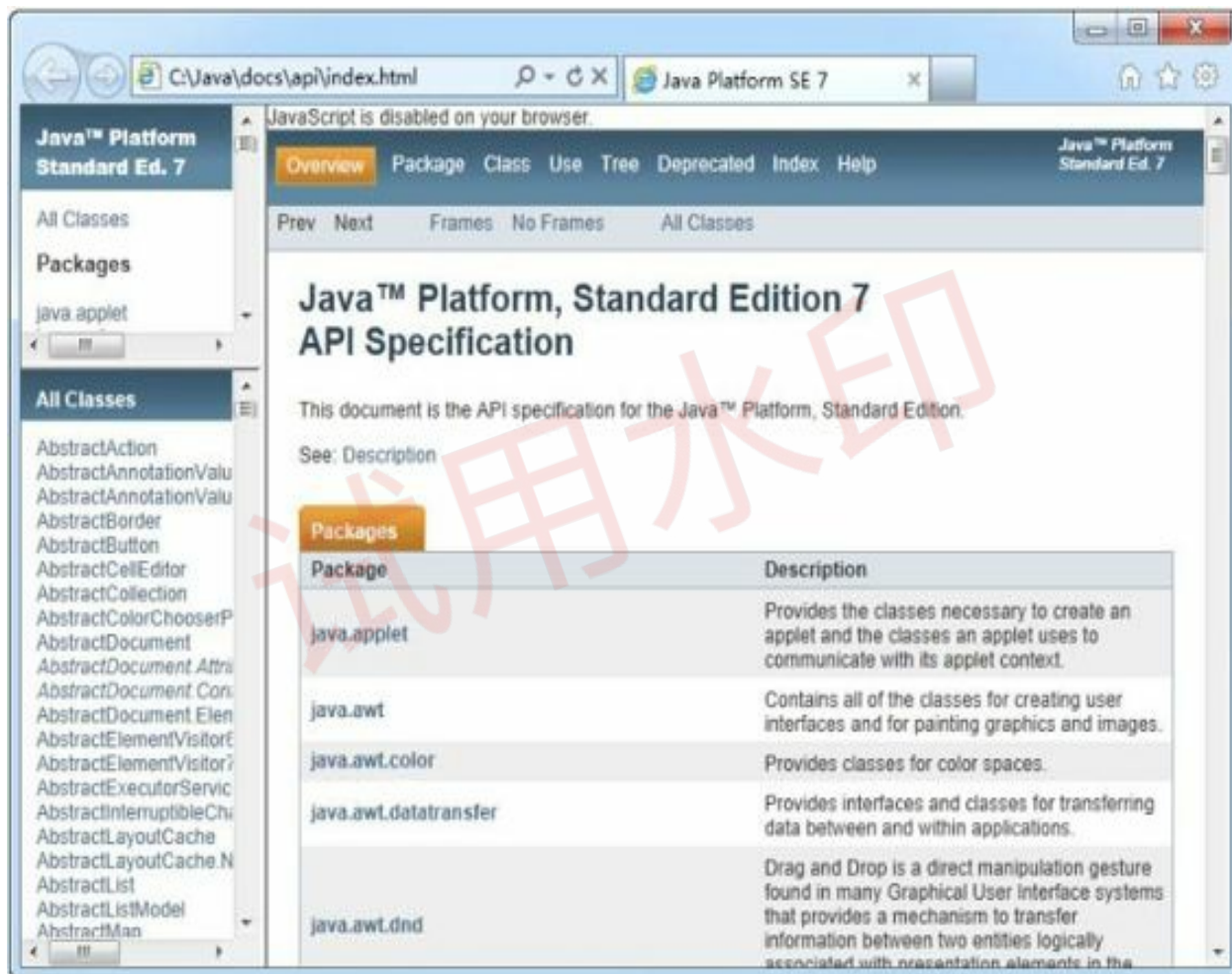


图1.5 Java API文档页面

1.2 Java语言的特性



视频讲解：光盘\TM\lx\1\Java语言的特性.exe

Java语言的作者们编写了具有广泛影响的Java白皮书，里面详细地

介绍了他们的设计目标以及实现成果，还用简短的篇幅介绍了Java语言的特性。下面将对这些特性进行扼要的介绍。

1.2.1 简单

Java语言的语法简单明了，容易掌握，而且是纯面向对象的语言。Java语言的简单性主要体现在以下几个方面：

- ☑ 语法规则和C++类似。从某种意义上讲，Java语言是由C和C++语言转变而来的，所以C程序设计人员可以很容易地掌握Java语言的语法。

- ☑ Java语言对C++进行了简化和提高。例如，Java使用接口取代了多重继承，并取消了指针，因为指针和多重继承通常使程序变得复杂。Java语言还通过实现垃圾自动收集，大大简化了程序设计人员的资源释放管理工作。

- ☑ Java提供了丰富的类库和API文档以及第三方开发包，另外还有大量的基于Java的开源项目，JDK（Java开发者工具箱）已经开放源代码，读者可以通过分析项目的源代码，从而提高自己的编程水平。

1.2.2 面向对象

面向对象是Java语言的基础，也是Java语言的重要特性，它本身就是一种纯面向对象的程序设计语言。Java提倡万物皆对象，语法中不能在类外面定义单独的数据和函数，也就是说，Java语言最外部的数据类型是对象，所有的元素都要通过类和对象来访问。

1.2.3 分布性

Java的分布性包括操作分布和数据分布，其中操作分布是指在多个不同的主机上布置相关操作，而数据分布是将数据分别存放在多个不同的主机上，这些主机是网络中的不同成员。Java可以凭借URL（统一资源定位符）对象访问网络对象，访问方式与访问本地系统相同。

1.2.4 可移植性

Java程序具有与体系结构无关的特性，可以方便地移植到网络上的不同计算机中。同时，Java的类库中也实现了针对不同平台的接口，使这些类库可以移植。

1.2.5 解释型

运行Java程序需要解释器。任何移植了Java解释器的计算机或其他设备都可以用Java字节码进行解释执行。字节码独立于平台，它本身携带了许多编译时的信息，使得连接过程更加简单，开发过程更加迅速，更具探索性。

1.2.6 安全性

Java语言删除了类似C语言中的指针和内存释放等语法，有效地避免了非法操作内存。Java程序代码要经过代码校验、指针校验等很多测试步骤才能够运行，所以未经允许的Java程序不可能出现损害系统平台的行为，而且使用Java可以编写防病毒和防修改的系统。

1.2.7 健壮性

Java程序的设计目标之一，是编写多方面的、可靠的应用程序，Java将检查程序在编译和运行时的错误，并消除错误。类型检查能帮助用户检查出许多在开发早期出现的错误。集成开发工具（如Eclipse、NetBeans）的出现也使编译和运行Java程序更加容易。

1.2.8 多线程

多线程机制能够使应用程序在同一时间并行执行多项任务，而且相应的同步机制可以保证不同线程能够正确地共享数据。使用多线程，可以带来更好的交互能力和实时行为。

1.2.9 高性能

Java编译后的字节码是在解释器中运行的，所以它的速度较多数交互式应用程序提高了很多。另外，字节码可以在程序运行时被翻译成特定平台的机器指令，从而进一步提高运行速度。

1.2.10 动态

Java在很多方面比C和C++更能够适应发展的环境，可以动态调整库中方法和增加变量，而客户端却不需要任何更改。在Java中进行动态

调整是非常简单和直接的。

1.3 搭建Java环境

“工欲善其事，必先利其器。”在学习Java语言之前，必须了解并搭建好它所需要的开发环境。要编译和执行Java程序，JDK（Java Developers Kits）是必备的。下面将具体介绍下载并安装JDK和配置环境变量的方法。

1.3.1 JDK下载



视频讲解：光盘\TM\lx\1\JDK下载.exe

Java的JDK又称Java SE（以前称J2SE），是Sun公司的产品，由于Sun公司已经被Oracle收购，因此JDK可以在Oracle公司的官方网站<http://www.oracle.com/index.html>下载。



注意

在Java 6出版之后，J2SE、J2EE和J2ME正式更名，将名称中的2去掉，更名后分别为Java SE、Java EE和Java ME。

下面以目前最新版本的JDK7为例介绍下载JDK的方法，具体步骤如下：

（1）打开IE浏览器，输入网址“<http://www.oracle.com/index.html>”，浏览Oracle官方主页。将光标移动到工具栏上的Downloads菜单项上，将显示下载列表下拉菜单，单击Java for Developers超链接，如图1.6所示。



图1.6 Oracle主页

(2) 将跳转到JDK的下载页面，在该页面中单击最新版本JDK的超链接，即如图1.7所示的Download按钮。在撰写本书时，最新的JDK版本为JDK7 Update 3。

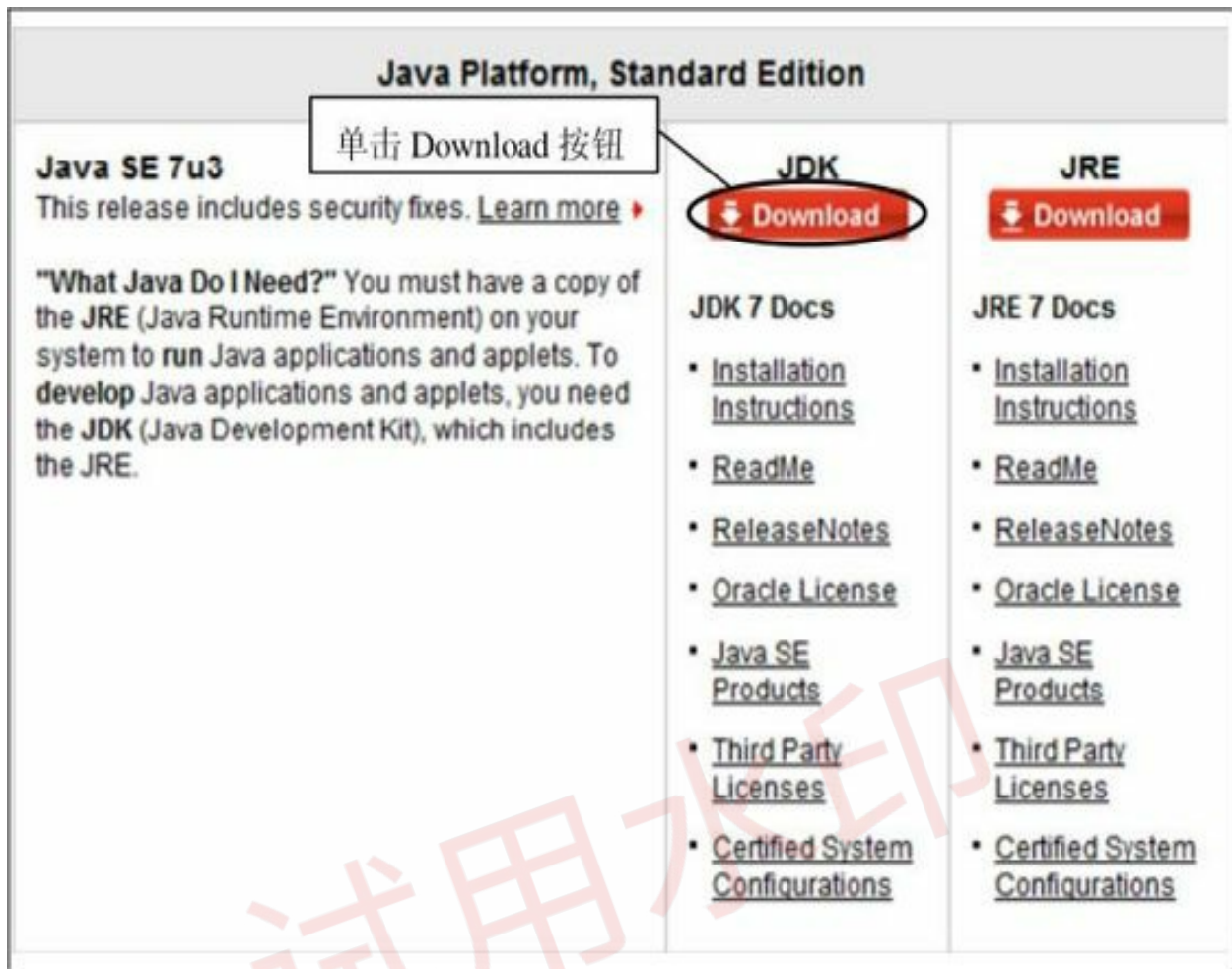


图1.7 JDK的下载页

(3) 在进入的新页面中，需要先选中同意协议的单选按钮，这时将显示如图1.8所示的页面，否则单击要下载的超链接时将不能进行下载。

Java SE Development Kit 7u3		
You must accept the Oracle Binary Code License Agreement for Java SE to download this software.		
Thank you for accepting the Oracle Binary Code License Agreement for Java SE; you may now download this software.		
Product / File Description	File Size	Download
Linux x86	63.65 MB	jdk-7u3-linux-i586.rpm
Linux x86	78.66 MB	jdk-7u3-linux-i586.tar.gz
Linux x64	64.53 MB	jdk-7u3-linux-x64.rpm
Linux x64	77.3 MB	jdk-7u3-linux-x64.tar.gz
Solaris x86	135.96 MB	jdk-7u3-solaris-i586.tar.Z
Solaris x86	81.4 MB	jdk-7u3-solaris-i586.tar.gz
Solaris SPARC	138.92 MB	jdk-7u3-solaris-sparc.tar.Z
Solaris SPARC		jdk-7u3-solaris-sparc.tar.gz
Solaris SPARC 64-bit		jdk-7u3-solaris-sparcv9.tar.Z
Solaris SPARC 64-bit		jdk-7u3-solaris-sparcv9.tar.gz
Solaris x64	14.46 MB	jdk-7u3-solaris-x64.tar.Z
Solaris x64	9.25 MB	jdk-7u3-solaris-x64.tar.gz
Windows x86	84.12 MB	jdk-7u3-windows-i586.exe
Windows x64	87.41 MB	jdk-7u3-windows-x64.exe

图1.8 JDK7 Update 3的下载列表



注意

下载时要选择适合自己操作系统平台的安装文件，如Windows系统平台是无法运行Linux系统平台的安装文件的。

(4) 在下载列表中，可以根据电脑硬件和系统选择适当的版本进行下载。如果是32位的Windows操作系统，那么需要下载jdk-7u3-windows-i586.exe文件，直接在页面单击该文件的超链接即可。

1.3.2 Windows系统的JDK环境



视频讲解：光盘\TM\lx\1\Windows系统的JDK环境.exe

1. JDK安装

下载Windows平台的JDK安装文件jdk-7u3-windows-i586.exe后即可安装，步骤如下：

（1）双击刚刚下载的安装文件，将弹出欢迎对话框，会要求接受Sun公司的许可协议，单击“接受”按钮，再单击“下一步”按钮，如图1.9所示。

（2）在弹出的“自定义安装”对话框中，可以选择安装的功能组件，这里选择默认设置，如图1.10所示。



图1.9 欢迎对话框

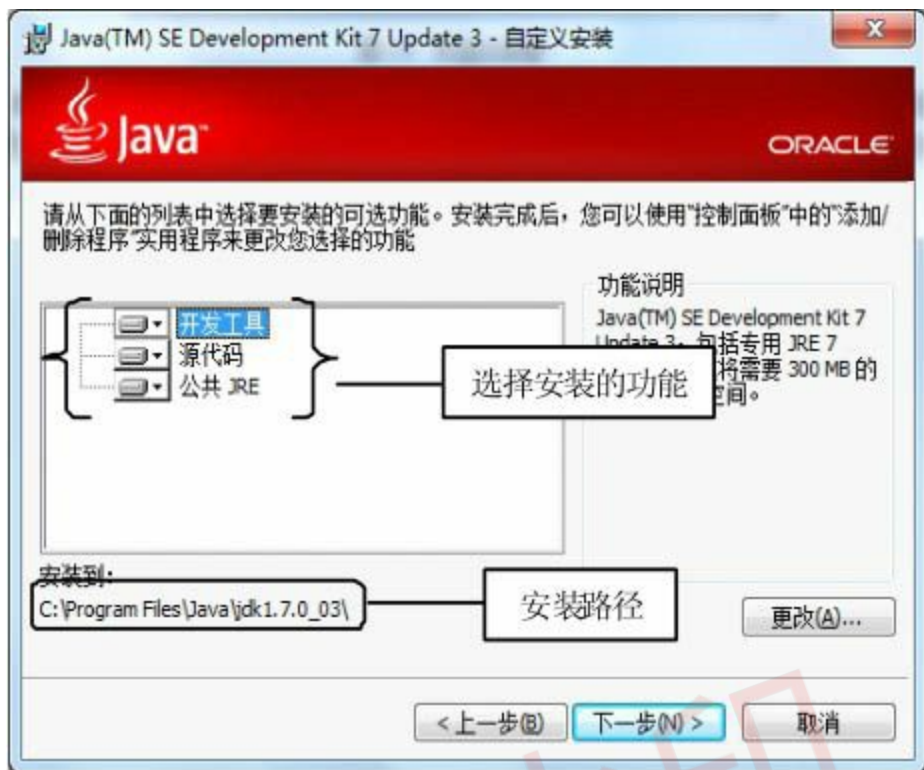


图1.10 “自定义安装”对话框

(3) 单击“更改”按钮，将弹出“更改文件夹”对话框，在该对话框中将JDK的安装路径更改为C:\Java\jdk1.7.0_03\，如图1.11所示，单击“确定”按钮，将返回到“自定义安装”对话框中。

(4) 单击“下一步”按钮，开始安装JDK。在安装过程中会弹出JRE的“目标文件夹”对话框，这里更改JRE的安装路径为C:\Java\jre7\，然后单击“下一步”按钮，安装向导会继续完成安装进程。



图1.11 更改JDK的安装路径对话框



说明

JRE全称为Java Runtime Environment，它是Java运行环境，主要负责Java程序的运行，而JDK包含了Java程序开发所需要的编译、调试等工具，另外还包含了JDK的源代码。

(5) 安装完成后，将弹出如图1.12所示的对话框，单击“继续”按钮，将安装JavaFX SDK。如果不想安装，可以单击“取消”按钮，取消JavaFX的安装。



图1.12 “完成”对话框



说明

JavaFX 2.0由甲骨文宣布推出, 是一款为企业业务应用提供的先进Java用户界面 (UI) 平台, 它能帮助开发人员无缝地实现与本地Java功能及Web技术动态能力的混合与匹配。

2. 在Windows 7系统中配置环境变量

在Windows 7系统中配置环境变量的步骤如下:

(1) 在“计算机”图标上单击鼠标右键, 在弹出的快捷菜单中选择“属性”命令, 在弹出的“属性”对话框左侧单击“高级系统设置”超链接, 将打开如图1.13所示的“系统属性”对话框。

(2) 单击“环境变量”按钮, 将弹出“环境变量”对话框, 如图1.14所示, 单击“系统变量”栏下的“新建”按钮, 创建新的系统变量。

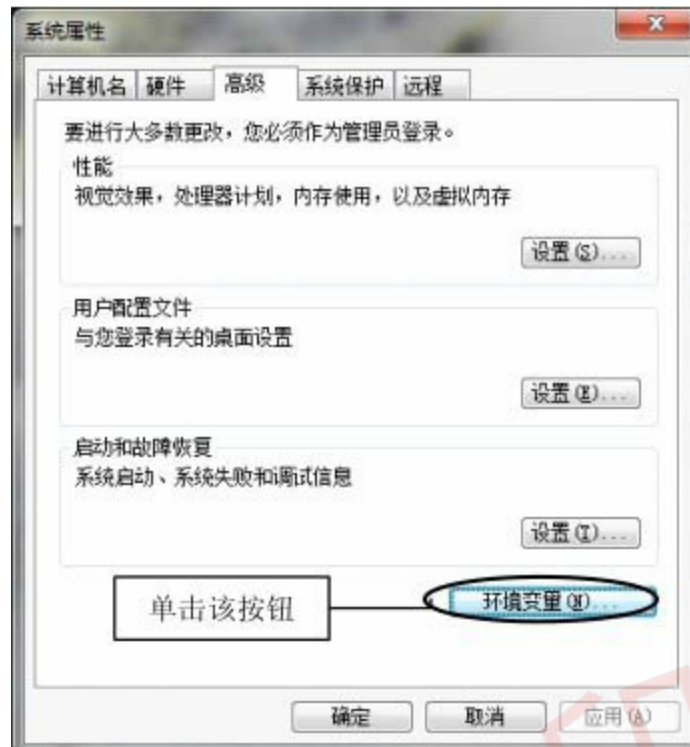


图1.13 “系统属性”对话框

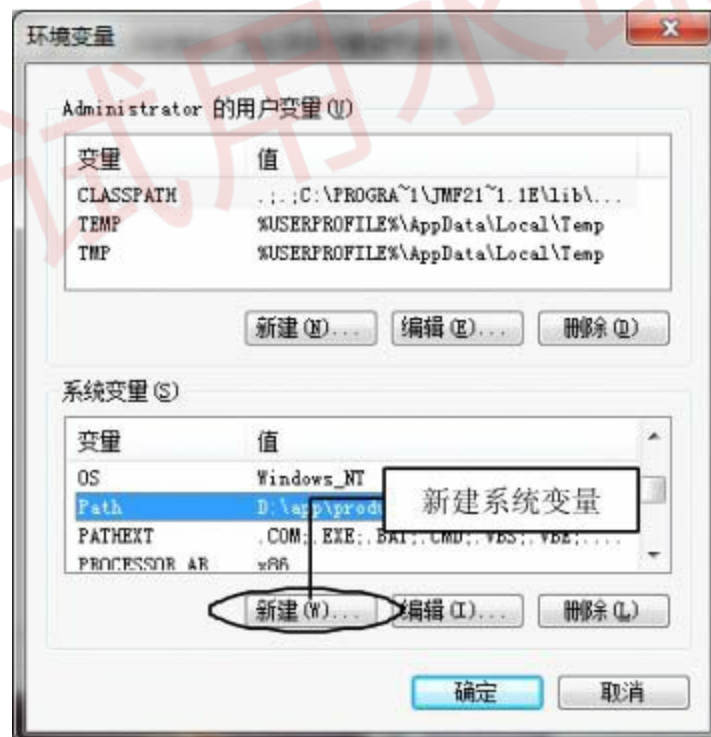


图1.14 “环境变量”对话框

(3) 弹出“新建系统变量”对话框，分别输入变量名“JAVA_HOME”和变量值（即JDK的安装路径），其中变量值是笔者

的JDK安装路径，读者需要根据自己的计算机环境进行修改，如图1.15所示。单击“确定”按钮，关闭“新建系统变量”对话框。

（4）在图1.14所示的“环境变量”对话框中双击Path变量对其进行修改，在原变量值最前端添加`;%JAVA_HOME%\bin;`变量值（注意，最后的“;”不要丢掉，它用于分割不同的变量值），如图1.16所示。单击“确定”按钮完成环境变量的设置。



图1.15 “新建系统变量”对话框



图1.16 设置Path环境变量值

（5）JDK安装成功之后必须确认环境配置是否正确。在Windows系统中测试JDK环境需要选择“开始” / “运行”命令（没有“运行”命令可以按Windows+R组合键），然后在“运行”对话框中输入“cmd”并单击“确定”按钮启动控制台。在控制台中输入javac命令，按Enter键，将输出如图1.17所示的JDK的编译器信息，其中包括修改命令的语法和参数选项等信息。这说明JDK环境搭建成功。

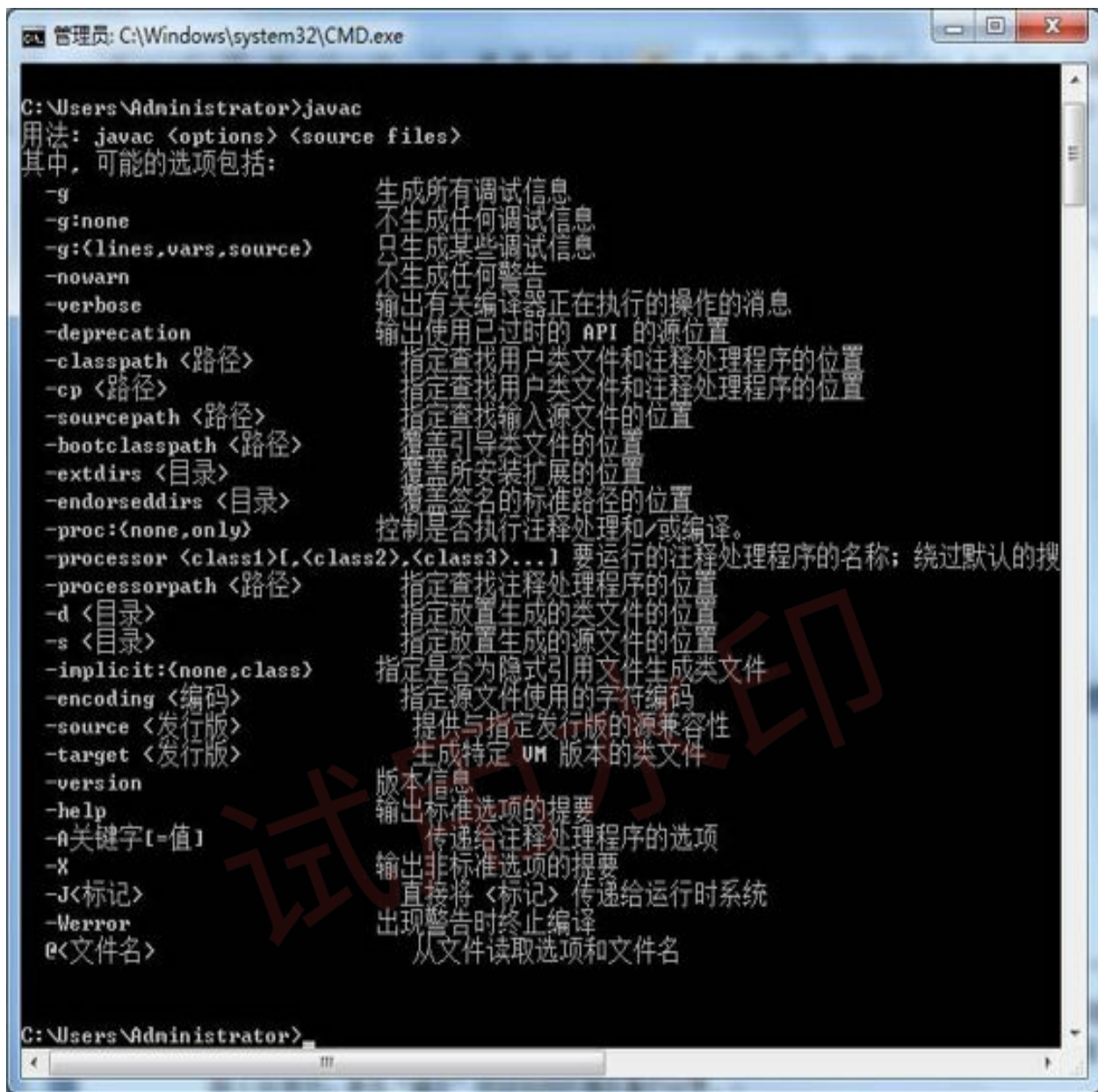


图1.17 JDK的编译器信息

3. 在Windows XP和Windows 2003系统中配置环境变量

在Windows XP和Windows 2003系统中配置环境变量的步骤如下:

(1) 在“我的电脑”图标上单击鼠标右键，在弹出的快捷菜单中选择“属性”命令，弹出“系统属性”对话框。选择“高级”选项卡，如图1.18所示，单击“环境变量”按钮，弹出“环境变量”对话框，如图1.19所示。单击“系统变量”下中的“新建”按钮，创建新的系统变量。

(2) 弹出“新建系统变量”对话框，分别输入变量

名“JAVA_HOME”和变量值“C:\Java\jdk1.7. 0_03”，其中变量值是笔者的JDK安装路径，读者需要根据自己的计算机环境进行修改，如图1.20所示。单击“确定”按钮，关闭“新建系统变量”对话框。

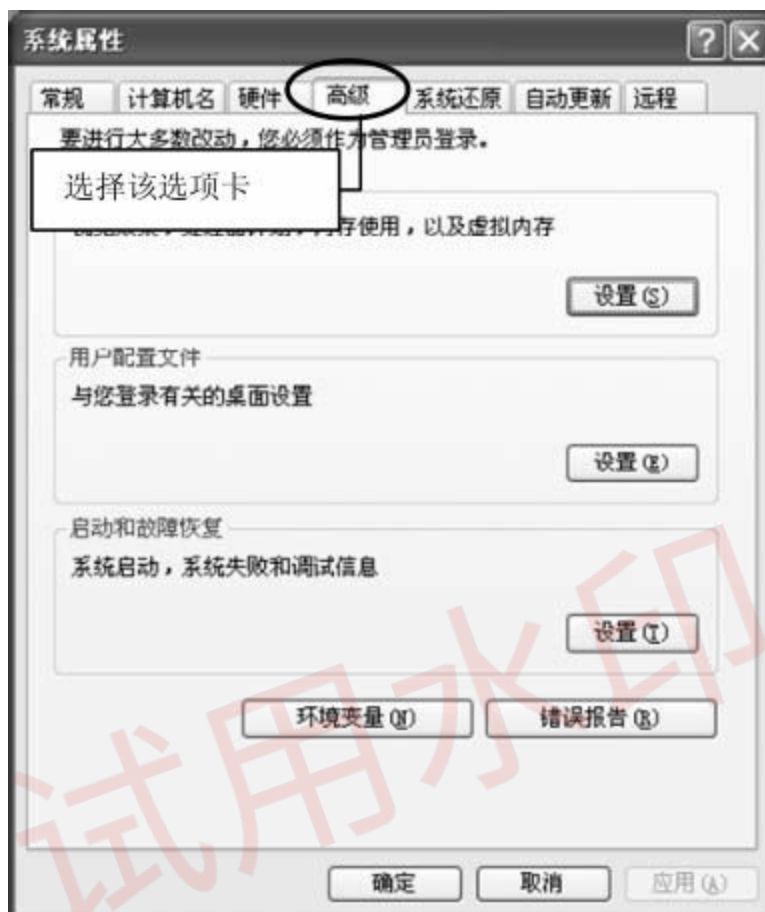


图1.18 “系统属性”对话框

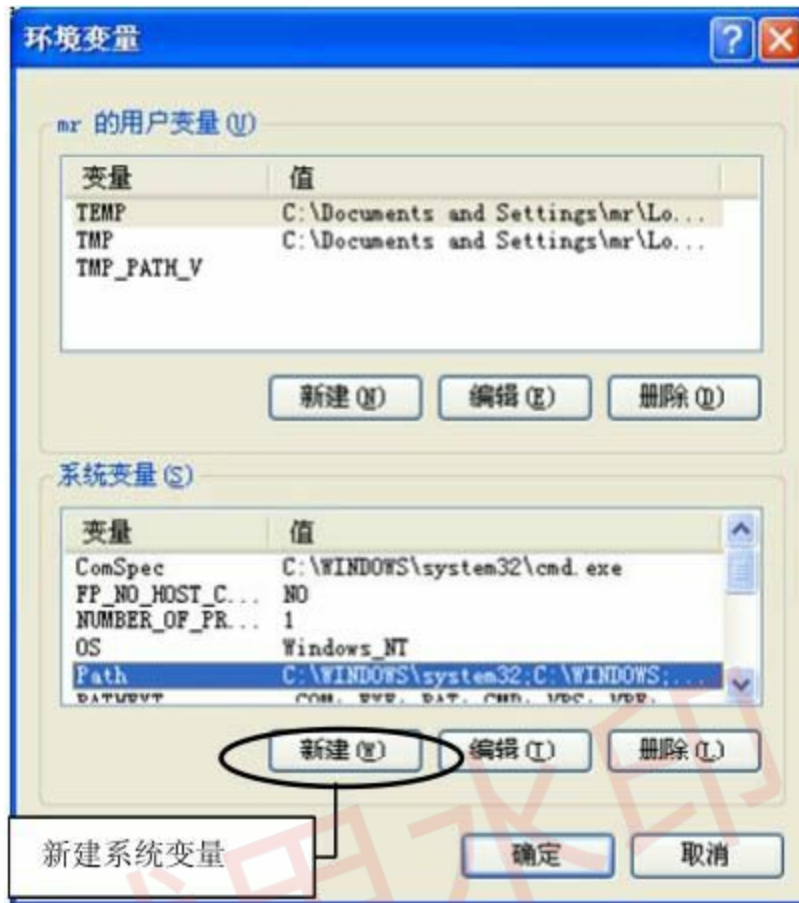


图1.19 “环境变量”对话框

(3) 在图1.19所示的“环境变量”对话框中双击Path变量对其进行修改，在原变量值之前添加;%JAVA_HOME%\bin;变量值（注意，最后的“;”不要丢掉，它用于分割不同的变量值），如图1.21所示。单击“确定”按钮完成环境变量的设置。



图1.20 “新建系统变量”对话框



图1.21 设置Path环境变量值

(4) JDK安装成功之后必须确认环境配置是否正确。在Windows系统中测试JDK环境需要选择“开始”/“运行”命令，然后在“运行”对话框中输入“cmd”并单击“确定”按钮启动控制台。在控制台中输入javac命令，按Enter键，将输出如图1.17所示的JDK的编译器信息，其中包括修改命令的语法和参数选项等信息。这说明JDK环境搭建成功。

1.4 第一个Java程序



视频讲解：光盘\TM\lx\1\第一个Java程序.exe

编写Java应用程序，可以使用任何一个文本编辑器来编写程序的源代码，然后使用JDK搭配的工具进行编译和运行。现在流行的开发工具可以自动完成Java程序的编译和运行，还带有代码辅助功能，可以提示完整的语法代码，但是大型的IDE开发工具需要的系统资源较大，在开发一个简单的程序时，还是原始的文本编辑器速度较快。本节将介绍使用文本编辑器开发一个简单Java程序的步骤。

【例1.1】下面编写本书的第一个Java程序，它在屏幕（也称控制台）上输出“HelloJava”信息。程序编写步骤如下：（实例位置：光盘\TM\sl\1.01）

(1) 使用文本编辑器编写Java程序代码的过程和平时编写文本文件是一样的，只需注意Java语法格式和编码规则即可。选择“开始”/“所有程序”/“附件”/“记事本”命令，在记事本程序中输入以下代码：

```
public class HelloJava {  
    public static void main(String[] args) {  
        System.out.println("Hello Java");  
    }  
}
```

(2) 选择“文件”/“保存”命令，选择存储位置为C盘根目录，在输入文件名称时，使用英文双引号（“ ”）把文件名称包含起来，

如“HelloJava.java”。这样可以防止记事本程序为文件自动添加.txt扩展名。

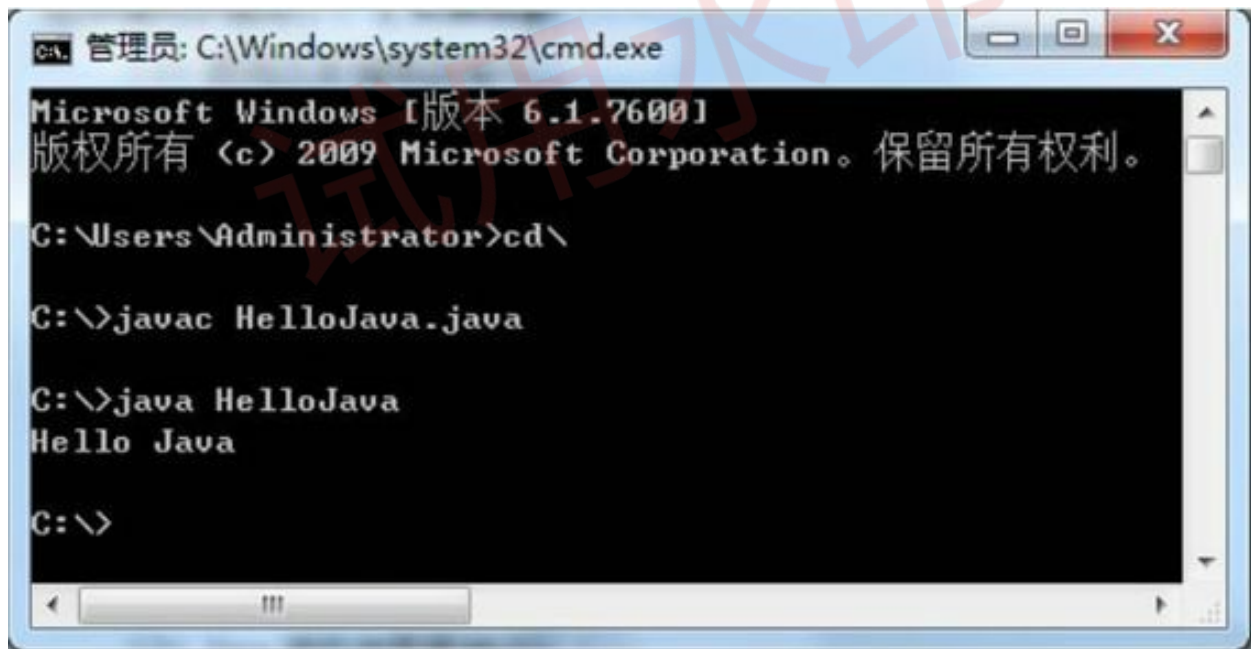
(3) Java源程序需要编译成字节码才能够被JVM识别，需要使用JDK的javac.exe命令。假设HelloJava.java文件保存在C盘，选择“开始”/“运行”命令，在“运行”对话框中输入“cmd”，单击“确定”按钮，启动控制台，在控制台中输入cd\命令将当前位置切换到C盘根目录，输入javac HelloJava.java命令编译源程序。源程序被编译后，会在相同的位置生成相应的.class文件，这是编译后的Java字节码文件。



注意

输入javac HelloJava.java命令时，要注意javac和HelloJava.java之间有一个空格符。如果没有输入这个空格符，将导致命令出错，无法执行。

(4) 在控制台中输入java HelloJava命令将执行编译后的HelloJava.class字节码文件。编译与运行Java程序的步骤以及运行结果如图1.22所示。



```
C:\Windows\system32\cmd.exe
Microsoft Windows [版本 6.1.7600]
版权所有 (c) 2009 Microsoft Corporation。保留所有权利。

C:\Users\Administrator>cd\

C:\>javac HelloJava.java

C:\>java HelloJava
Hello Java

C:\>
```

图1.22 编译与运行Java程序的步骤以及运行结果



注意

使用Javac××.java命令编译Java源程序时，如果没有弹出错误提示信息，说明编译成功，也许会提示一些警告信息，但编译也能够通过；但是如果出现Exception类的异常错误信息，则

说明源程序的代码有问题，无法完成编译过程，这时可以根据相应的Exception异常判断错误原因和代码位置，解决代码错误。有关异常的知识将在本书的第12章中介绍。

1.5 小结

本章简单介绍了Java语言及其相关特性，另外还分别介绍了在Windows系统平台中搭建Java环境的方法，以及编写Java程序的简单步骤。通过本章的学习，读者应该能够了解什么是Java和它的不同版本以及如何学习Java语言。搭建Java环境是本章的重点，读者应该熟练掌握。

1.6 实践与练习

1. 尝试修改例1.1的实例，使程序输出“这是我的第一个Java程序”。（答案位置：光盘\TM\sl\1.02）
2. 尝试编写Java程序，使程序输出如图1.23所示的运行结果。（答案位置：光盘\TM\sl\1.03）

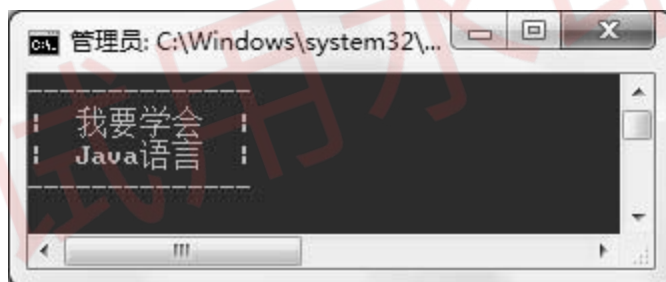


图1.23 Java程序运行结果

第2章

熟悉Eclipse开发工具

( 视频讲解：35分钟)

学习Java语言程序设计必须选择一个功能强大、使用简单、能够辅助程序设计的IDE（集成开发工具）。Eclipse是目前最流行的Java语言开发工具，它强大的代码辅助功能，帮助程序开发人员自动完成语法修正、补全文字、代码修正、API提示等编码工作，大量节省程序开发所需的时间和精力。本章将简要介绍Eclipse开发工具，使读者能够初步了解Eclipse并使用它完成程序设计工作。

通过阅读本章，您可以：

- ▶▶ 掌握安装Eclipse的国际化语言包
- ▶▶ 学习Eclipse中经常使用的菜单和工具栏
- ▶▶ 学习使用Eclipse编写程序代码
- ▶▶ 掌握Eclipse中调试器的使用

2.1 熟悉Eclipse

虽然使用记事本和JDK编译工具已经可以编写Java程序，但是在项目开发过程中必须使用大型的集成开发工具（IDE）来编写Java程序，这样可以避免编码错误，方便管理项目结构，而且使用IDE工具的代码辅助功能可以快速地输入程序代码。本节将介绍Eclipse开发工具，包括它的安装、配置与启动、菜单栏、工具栏以及各种视图的作用等。

2.1.1 Eclipse简介



视频讲解：光盘\TM\lx\2\Eclipse简介.exe

Eclipse是由IBM公司投资4000万美元开发的集成开发工具。它基于Java语言编写，并且是开放源代码的、可扩展的，也是目前最流行的Java集成开发工具之一。另外，IBM公司捐出Eclipse源代码，组建了Eclipse联盟，由该联盟负责这种工具的后续开发。Eclipse为编程人员提供了一流的Java程序开发环境，它的平台体系结构是在插件概念的基础上构建的，插件是Eclipse平台最具特色的特征之一，也是其区别于其他开发工具的特征之一。学习了本章之后，读者将对Eclipse有一个初步的了解，为后面深入学习做准备。



说明

本书所使用的Eclipse开发工具的版本是3.7，读者可以从<http://www.eclipse.org>网站下载。同时，该网站的Babel项目组提供了Eclipse 3.7的多国语言包，它能够在任何国家或地区以本地语言显示程序界面，其中包括菜单栏、工具栏、窗体和所有程序模块（不包括其他插件）。

2.1.2 下载Eclipse



视频讲解：光盘\TM\lx\2\下载Eclipse.exe

本节将介绍如何到Eclipse的官方网站下载本书所使用的Eclipse开发环境。掌握Eclipse的下载与使用，并不只是为了学习，以后工作中Eclipse也是程序开发的好帮手。事实上，Eclipse已经成为使用最广泛、应用最多的Java开发工具，并且是由Java语言编写的。其下载步骤如下：

（1）打开浏览器，在地址栏中输入“<http://www.eclipse.org>”后，按Enter键开始访问Eclipse的官方网站，该网站的首页面包含了下载超链接，如图2.1所示。单击页面上的Downloads菜单项或者右侧的Download Eclipse超链接都可以进入到下载页面。

（2）Eclipse下载页面中包含各种版本的Eclipse下载区域，其中第3个栏目是Java开发版的Eclipse（它包含Java IDE、CVS客户端、XML编辑器和WindowBuilder等），在每个栏目的右侧是各种平台的下载超链接，本书使用的是32位的Windows平台，所以单击Windows 32 Bit超链接，如图2.2所示。

（3）最后的Eclipse下载页面会根据客户端所在的地理位置，分配合理的下载镜像站点，用户只需在Eclipse下载页面中单击绿色箭头即可下载Eclipse文件，如图2.3所示。



图2.1 Eclipse网站首页面



图2.2 Eclipse下载页面



图2.3 最后的Eclipse下载页面

2.1.3 安装Eclipse的中文语言包



视频讲解：光盘\TM\1\2\安装Eclipse的中文语言包.exe

从网站中下载的Eclipse安装文件是一个压缩包，将其解压缩到指定的文件夹，然后运行文件夹中的Eclipse.exe文件，即可启动Eclipse开发工具。但是在启动Eclipse之前需要安装中文语言包，以降低读者的学习难度。

Eclipse的国际语言包可以到<http://www.eclipse.org/babel>下载，具体的下载和使用步骤如下：

(1) 在浏览器的地址栏中输入“<http://www.eclipse.org/babel>”，并按Enter键，进入到如图2.4所示的Babel项目组首页。

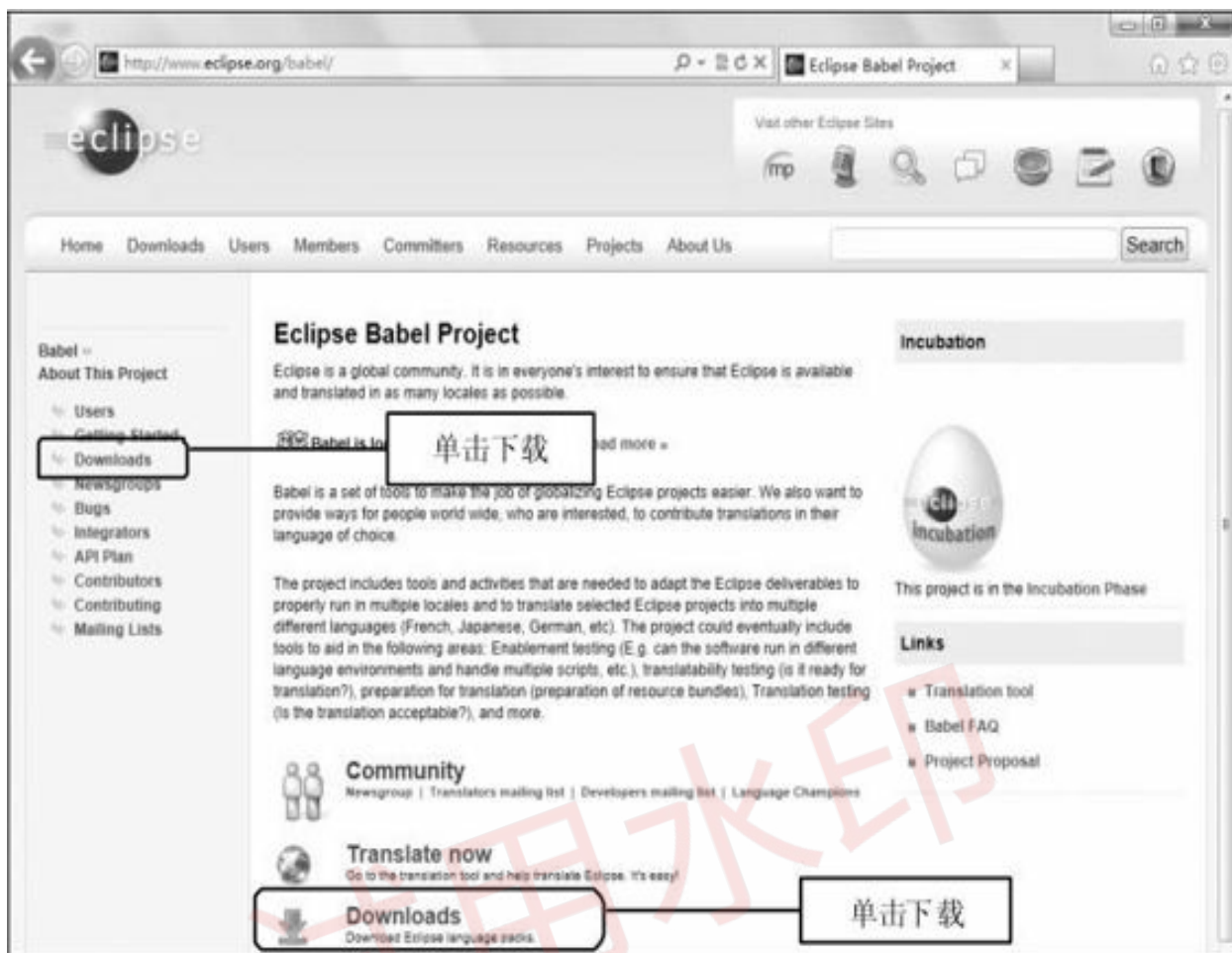


图2.4 Babel项目组首页

(2) 单击页面左侧导航中的Downloads超链接或者单击页面下方的绿色箭头都可以进入到语言包的下载页面。



注意

下载Eclipse多国语言包时，一定要注意语言包所匹配的Eclipse版本，否则可能无法实现Eclipse的国际化。

(3) 在下载页面的Babel Language Packs标题下选择对应Eclipse版本的超链接下载语言包，Eclipse 3.7的名称是Indigo，所以单击该超链接，如图2.5所示，进入Eclipse 3.7的Babel语言包下载页面，在该页面中包含了对应各国语言的资源包，而每个语言的资源包又按插件与功能模块分为多个zip压缩包。

Eclipse Babel Project Downloads

Babel Language Packs R0.9.1 are now available!

Babel Language Packs R0.9.1 are now available! Translations in all languages for all projects in Babel up to November 28th, 2011 are included in R0.9.1.

Babel is currently setup to accept translation contributions in 40 languages for about 40+ Eclipse projects and sub-projects. Participation for each language and each project may vary. For some of the most common projects, the completion percentages are very high. For example, the Eclipse 3.7 project has 4 languages that are 90%+ complete, 23 languages that are 70%+ complete. See the Babel Translation Statistics for more details.

All downloads are provided under the terms and conditions of the Eclipse Foundation Software User Agreement unless otherwise specified.

The Babel project produces language packs from the translations entered using the Babel Translation Tool. You can download the Babel Language Pack Zips from the following links, or use the Babel Language Pack Update Sites to download the languages packs.

Babel Language Pack Zips and Update Sites - R0.9.1 (2011/11/28)

Babel Language Pack Zips

Indigo | Helios | Galileo 选择合适的版本

Babel Language Pack Update Site for Indigo
<http://download.eclipse.org/technology/babel/update-site/R0.9.1/indigo>

Babel Language Pack Update Site for Helios
<http://download.eclipse.org/technology/babel/update-site/R0.9.1/helios>

Babel Language Pack Update Site for Galileo
<http://download.eclipse.org/technology/babel/update-site/R0.9.1/galileo>

图2.5 Babel语言包下载页面

(4) 在页面中找到简体中文的语言包分类，如图2.6所示，可以单独选择Eclipse的语言包下载，也可以下载全部语言包。例如，单独下载Eclipse的语言包，可以单击BabelLanguagePack-eclipse-zh_3.7.0.v20111128043401.zip(87.36%)超链接，下载后的文件名为BabelLanguagePack-eclipse-zh_3.7.0.v2011 1128043401.zip。

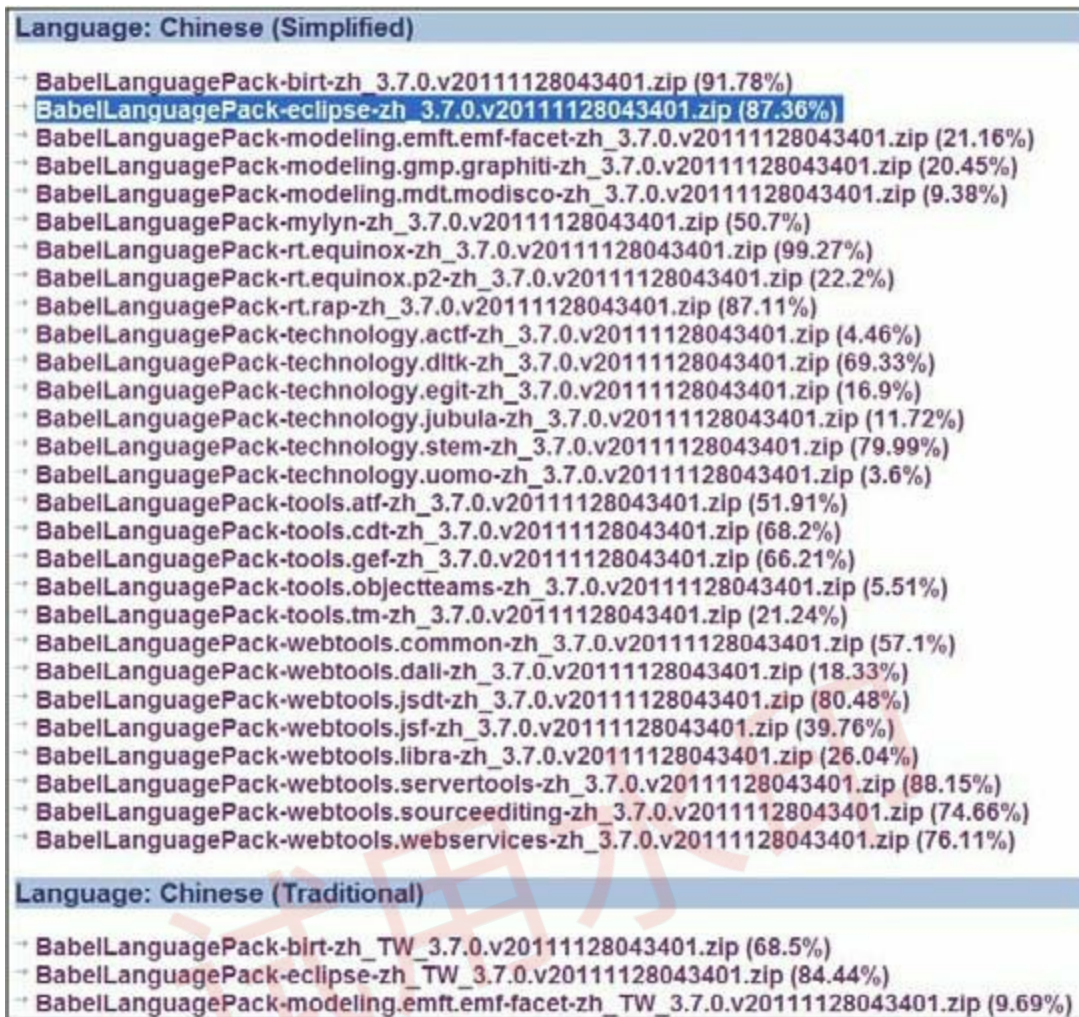


图2.6 中文语言包下载分类

(5) 将下载的所有语言包解压缩并覆盖Eclipse文件夹中同名的两个文件夹features和plugins，这样在启动Eclipse时便会自动加载语言包。

2.1.4 Eclipse的配置与启动



视频讲解：光盘\TM\lx\2\Eclipse的配置与启动.exe

现在已经配置好Eclipse的多国语言包，可以启动Eclipse了。在Eclipse的安装文件夹中运行eclipse.exe文件，即开始启动Eclipse，弹出“工作空间启动程序”对话框，该对话框用于设置Eclipse的工作空间（工作空间用于保存Eclipse建立的程序项目和相关设置）。本书的开发环境统一设置工作空间为Eclipse安装位置的workspace文件夹，在“工作空间启动程序”对话框的“工作空间”文本框中输入“.\workspace”，单击“确定”按钮，即可启动Eclipse，如图2.7所示。

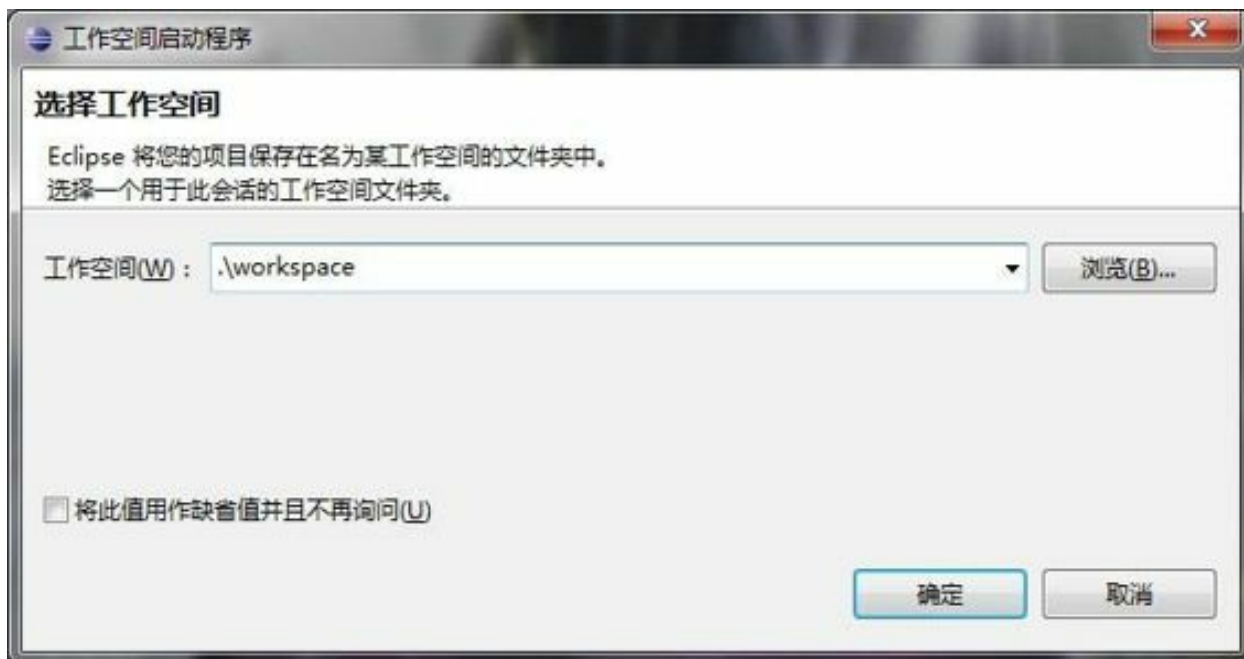


图2.7 设置工作空间



注意

每次启动Eclipse时都会出现设置工作空间的对话框，通过选中“将此值用作缺省值并且不再询问”复选框可以设置默认工作空间，这样Eclipse启动时就不会再询问工作空间的设置了。



技巧

如果在启动Eclipse时，选中“将此值用作缺省值并且不再询问”复选框，设置不再询问工作空间设置后，可以通过以下方法恢复提示。首先选择“窗口”/“首选项”命令，打开“首选项”对话框，然后在左侧选择“常规”/“启动和关闭”/“工作空间”节点，并且将右侧的“启动时提示工作空间”复选框选中，单击“应用”按钮后，再单击“确定”按钮即可。

Eclipse首次启动时，会显示Eclipse欢迎界面，其中包括Eclipse的概述、新增内容、样本、教程以及工作台的相关按钮，如图2.8所示。



图2.8 Eclipse的欢迎界面

2.1.5 Eclipse工作台



视频讲解：光盘\TM\lx\2\Eclipse工作台.exe

在Eclipse的欢迎界面中，单击“工作台”（Workbench）按钮或关闭欢迎界面，将显示Eclipse的工作台，它是程序开发人员开发程序的主要场所。Eclipse还可以将各种插件无缝地集成到工作台中，也可以在工作台中开发各种插件。Eclipse工作台主要包括标题栏、菜单栏、工具栏、编辑器、透视图和相关的视图等，如图2.9所示。在接下来的章节中将介绍Eclipse的菜单栏与工具栏，以及什么是透视图、视图，并介绍常用视图。

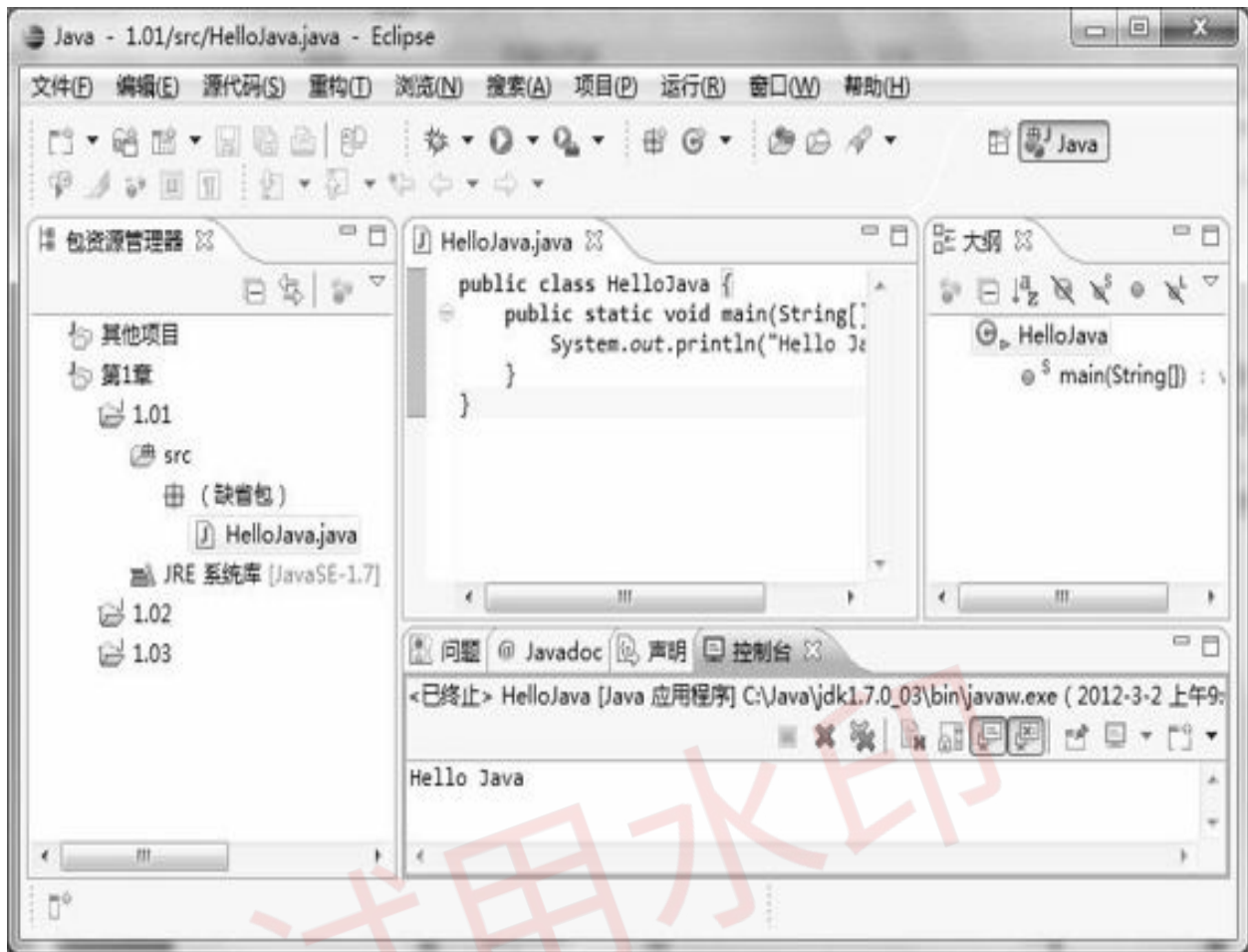


图2.9 Eclipse工作台

2.1.6 透视图与视图



视频讲解：光盘\TM\lx\2\透视图与视图.exe

透视图和视图是Eclipse中的概念，本节将分别介绍什么是透视图、视图及其在Eclipse中的作用。

1. 透视图

透视图是Eclipse工作台提供的附加组织层，它实现多个视图的布局 and 可用操作的集合，并为这个集合定义一个名称，起到一个组织的作用。例如，Eclipse提供的Java透视图组织了与Java程序设计有关的视图和操作的集合，而“调试”透视图负责组织与程序调试有关的视图和操作集。在Eclipse的Java开发环境中提供了几种常用的透视图，如Java透视图、“资源”透视图、“调试”透视图、“小组同步”透视图等。不同的透视

图之间可以进行切换，但是同一时刻只能使用一个透视图。

2. 视图

视图多用于浏览信息的层次结构和显示活动编辑器的属性，例如，“控制台”视图用于显示程序运行时的输出信息和异常错误，而“包资源管理器”视图可以浏览项目的文件组织结构。视图可以单独出现，也可以与其他视图以选项卡样式叠加在一起，它们可以有自己独立的菜单和工具栏，并且可以通过拖动随意改变布局位置。



技巧

在视图标题上单击鼠标右键，在弹出的快捷菜单中选择“已拆离”命令可以将视图从Eclipse工作台界面中分离，以单独窗口存在，同样的操作也可以将视图再次合并到工作台界面中。本书大部分视图都是从Eclipse中分离后截取的界面效果。

2.1.7 菜单栏



视频讲解：光盘\TM\lx\2\菜单栏.exe

Eclipse的菜单栏包含了Eclipse的基本命令，在使用不同的编辑器时，还会动态地添加有关该编辑器的菜单。基本的菜单栏中除了常用的“文件”、“编辑”、“窗口”和“帮助”等菜单以外，还提供了一些功能菜单，如“源代码”和“重构”等，如图2.10所示。



图2.10 Eclipse的菜单栏

每个菜单中都包含不同的命令，这些命令用于完成最终的操作，如文件的打开与保存、代码格式化、程序的运行与分步调试等。每个菜单所包含的命令如图2.11所示。

文件(F) 新建(N) Alt+Shift+N 打开文件... 关闭(C) Ctrl+W 全部关闭(L) Ctrl+Shift+W 保存(S) Ctrl+S 另存为(A)... 全部保存(Q) Ctrl+Shift+S 还原(R) 移动(M) 重命名(O) 删除(D) F5 将行定界符转换为... 打印(P) Ctrl+P 切换工作空间(S)... 导入(I)... 导出(E)... 属性(A) Alt+Enter 退出(X)	编辑(E) 撤销(U) 类型化 Ctrl+Z 重做(O) Ctrl+Y 剪切(X) Ctrl+X 复制(C) Ctrl+C 复制指定名(N) 粘贴(V) Ctrl+V 删除(D) Delete 全部选中(A) Ctrl+A 将选择范围扩展到(E) 查找/替换(F) Ctrl+F 查找下一个(N) Ctrl+N 查找上一个(P) Ctrl+Shift+N 增量式查找下一个(G) Ctrl+J 增量式查找上一个(B) Ctrl+Shift+J 添加书签(B)... 添加任务(T)... 灵活插入方式(W) Ctrl+Shift+Insert 显示工具提示描述(H) F2 内容辅助(A) 文字补全(C) Ctrl+Alt+/ 快速修正(Q) Ctrl+; 设置编码(E)...	源代码(S) 切换注释(C) Ctrl+/ 添加块注释(B) Ctrl+Shift+/ 删除块注释(D) Ctrl+Shift+\ 生成元素注释(E) Alt+Shift+J 右移(R) 左移(L) 更正缩进(I) Ctrl+I 格式(F) Ctrl+Shift+F 格式化元素(E) 添加导入(I) Ctrl+Shift+M 组织导入(O) Ctrl+Shift+O 对成员排序(S)... 清理(C)... 覆盖/实现方法(O)... 生成 Getter 和 Setter... 生成代理方法(M)... 生成 hashCode() 和 equals()... 使用手段生成构造函数(G)... 从超类中生成构造函数(C)... 包的方式(W) Alt+Shift+S 外部化字符串(E)... 查找错误的内部化字符串(E)	重构(R) 重命名(O) Alt+Shift+R 移动(M) Alt+Shift+V 更改方法特征符(C) Alt+Shift+C 抽取方法(M) Alt+Shift+B 抽取局部变量(L) Alt+Shift+L 抽取常量(A)... 内联(I) Alt+Shift+I 将匿名类转换为嵌套类(N)... 将成员类型转换为顶级(T) 将局部变量转换为字段(F)... 抽取超类(C)... 抽取接口(I)... 尽可能使用超类型(S)... 下推(D)... 上拉(U)... 引入抽象(A)... 引入工厂(F)... 引入参数(P)... 包裹字段(E)... 通用化已声明的类型(G)... 推断通用类型参数(G)... 迁移 Javadoc 文件... 创建脚本(S)... 应用脚本(S)... 历史记录(Q)...
帮助(H) 欢迎(W) 帮助内容(O) 搜索(S) Ctrl+H 动态帮助(D) 编辑帮助(E) Ctrl+Shift+L 提示和技巧(T)... 备忘录(N)... 软件更新(U) 关于 Eclipse SDK(A)	搜索(S) 搜索(A) Ctrl+H 文件(F)... Java... 文本(T) 引用(R) 声明(D) 实现(I) 访问(A) 访问(W) 文件中的出现位置(L) 引用测试(T)... 项目(P) 打开项目(O) 关闭项目(C) 全部构建(C) Ctrl+B 构建项目(B) 构建工作集(W) 清理(C)... 自动构建(A) 生成 Javadoc(G)... 属性(A)	运行(R) 运行上次启动(E) Ctrl+F11 调试上次启动(D) F11 运行历史记录(H) 运行方式(W) 运行(R) 调试历史记录(H) 调试方式(W) 调试(T)... 查看(V) 检查(C) Ctrl+Shift+I 显示(S) Ctrl+Shift+D 执行(E) Ctrl+R 单步跳入选择的内容(E) Ctrl+F5 切换断点(B) Ctrl+Shift+B 切换方法断点(M) 切换观察点(O) 跳过的所有断点(O) 删除所有断点(D) 添加 Java 异常断点(E)... 添加类加载断点(E)... 外部工具(E)	浏览(B) 进入(E) 转至(T) 打开声明(D) F3 打开类型层次结构(H) F4 打开调用层次结构(C) Ctrl+Alt+H 打开超类实现(S) 打开外部的 Javadoc Shift+F2 打开类型(T) Ctrl+Shift+T 在层次结构中打开类型(C) Ctrl+Shift+H 打开资源(R) Ctrl+Shift+R 显示位置(L) Alt+Shift+F 快速大纲(O) Ctrl+O 快速类型层次结构(C) Ctrl+T 下一个注释(N) Ctrl+. 上一个注释(P) Ctrl+, 上一个编辑位置(C) Ctrl+Q 转至行(G) Ctrl+L 后退(B) Alt+左箭头 前进(F) Alt+右箭头

图2.11 Eclipse菜单命令

菜单中的命令虽多，但不是所有命令都经常使用。本节将介绍几个最常用的菜单及其命令，其他不常用的菜单，读者可以在日后程序开发过程中慢慢掌握。

1. “文件”菜单

“文件”菜单中包含“新建”、“保存”、“关闭”、“打印”、“切换工作空间”、“属性”等命令。菜单中包含的内容虽多，但也有常用的和不常用的，例如，如果不常使用打印功能，那么“文件”菜单中的“打印”命令会很少用到。“文件”菜单中的常用命令列于表2.1中，这样阅读和查询更方便。

表2.1 “文件”菜单中常用命令

图 标	命 令	说 明	快 捷 键
	新建	创建新项目、元素或资源	Alt +Shift+N
	打开文件	打开已经存在的文件	
	关闭	关闭当前编辑器	Ctrl+W
	全部关闭	关闭所有编辑器	Ctrl+Shift+W
	保存	保存当前编辑器的内容	Ctrl+S
	刷新	刷新所选元素的内容	F5
	切换工作空间	切换工作空间到其他位置。这将导致 Eclipse 重启	
	导入	打开导入向导对话框	
	导出	打开导出向导对话框	
	属性	打开所选元素的属性对话框	Alt+Enter

2. “编辑”菜单

“编辑”菜单用于辅助程序代码设计工作，除常用的“剪切”、“复制”、“粘贴”命令之外，还提供了“快速修正”、“将选择范围扩展到”和“内容辅助”等高级命令。“编辑”菜单中的常用命令如表2.2所示。

表2.2 “编辑”菜单中常用命令

命 令	说 明	快 捷 键
将选择范围扩展到	将选择编辑内容的范围扩大到外层元素、下一个元素、上一个元素或者恢复上一次选择的元素	
查找/替换	搜索编辑器中的内容片段，并根据需要替换为新的内容	Ctrl+F
查找下一个	搜索当前所选内容下一次出现的地方	Ctrl+K
查找上一个	搜索当前所选内容上一次出现的地方	Ctrl+Shift+K
添加书签	在当前光标所在行添加书签	
添加任务	在当前光标所在行添加任务	
灵活插入方式	切换插入方式。当禁用灵活插入方式时，将禁用自动缩进、添加右方括号等辅助功能	Ctrl+Shift+Insert
内容辅助	在当前光标位置打开内容辅助对话框	
文字补全	补全当前编辑器中正在输入的文字	Ctrl+Alt+/
快速修正	如果光标位于问题代码附近，则打开一个解决方案对话框	Ctrl+l

3. “源代码”菜单

“源代码”菜单中包含的命令都是和代码编写相关的命令，主要用于辅助编程。“源代码”菜单的常用命令如表2.3所示。

表2.3 “源代码”菜单中常用命令

命 令	说 明	快 捷 键
切换注释	注释或取消注释当前选择的所有行	Ctrl+/或 Ctrl+7
添加块注释	在当前选择的多行代码周围添加块注释	Ctrl+Shift+/
除去块注释	从当前选择的多行代码中除去块注释	Ctrl+Shift+\
更正缩进	更正当前选择的代码行的缩进	Ctrl+I
格式	使用代码格式化程序来格式化当前 Java 代码	Ctrl+Shift+F
组织导入	导入当前类所使用的类包	Ctrl+Shift+O
覆盖/实现方法	使用向导覆盖父类或实现接口中的方法	
生成 Getter 和 Setter	使用向导创建成员变量的 getXXX()/setXXX()方法	
生成 hashCode()和 equals()	打开“生成 hashCode()和 Equals()”对话框	
使用字段生成构造函数	添加构造函数，这些构造函数初始化当前所选类型的字段。可用于类型、字段或类型中的文本选择	
从超类中生成构造函数	对于当前所选类型，按照超类中的定义来添加构造函数	
包围方式	使用代码模板包围所选语句	Alt+Shift+Z
外部化字符串	打开“将字符串外部化”向导，此向导允许通过使用语句访问属性文件来替换代码中的所有字符串	

4. “重构”菜单

“重构”菜单是Eclipse最关键的菜单，主要包括项目重构的相关命令，应该重点掌握。“重构”菜单中的常用命令如表2.4所示。

表2.4 “重构”菜单中常用命令

命 令	说 明	快 捷 键
重命名	重命名所选择的 Java 元素	Alt+Shift+R
移动	移动所选择的 Java 元素	Alt+Shift+V
抽取方法	创建一个包含当前所选语句或表达式的新方法，并相关地引用	Alt+Shift+M
抽取局部变量	创建为当前所选表达式指定的新变量，并将选择替换为对新变量的引用	Alt+Shift+L
抽取常量	从所选表达式创建静态终态字段并替换字段引用，并且可以选择重写同一表达式的其他出现位置	
内联	直接插入局部变量、方法或常量	Alt+Shift+I
将匿名类转换为嵌套类	将匿名内部类转换为成员类	
将成员类型转换为顶级	为所选成员类型创建新的 Java 编译单元，并根据需要更新所有引用	
将局部变量转换为字段	如果该变量是在创建时初始化的，则此操作将把初始化移至新字段的声明或类的构造函数	
抽取超类	从一组同类型中抽取公共超类	
抽取接口	根据当前类的方法创建接口，并使该类实现这个接口	
包括字段	将对变量的所有引用替换为 getXXX()/setXXX()方法	
历史记录	浏览工作空间重构历史记录，并提供用于从重构历史记录中删除重构的选项	

2.1.8 工具栏



视频讲解：光盘\TM\lx\2\工具栏.exe

和大多数软件的布局格式相同，Eclipse的工具栏位于菜单栏的下方。工具栏中的按钮都是菜单命令对应的快捷图标，在打开不同的编辑器时，还会动态地添加与编辑器相关的新工具栏按钮。另外，除了菜单栏下面的主工具栏，Eclipse中还有视图工具栏、透视图工具栏和快速视图工具栏等多种工具栏。

☒ 主工具栏

主工具栏就是位于Eclipse菜单栏下方的工具栏，其内容将根据不同的透视图和不同类型的编辑器显示相关工具按钮，如图2.12所示。

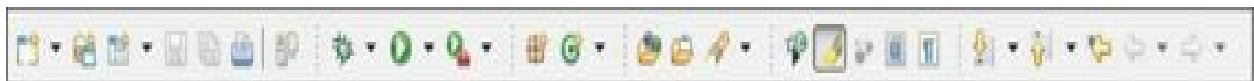


图2.12 Eclipse主工具栏

☒ 视图工具栏

Eclipse界面中包含多种视图，这些视图有不同的用途（有关视图的概念已在2.1.6节中讲述），可以根据视图的功能需求在视图的标题栏位置添加相应的视图工具栏。例如，“控制台”视图用于输出程序运行中的输出结果和运行时异常信息，其工具栏如图2.13所示。



图2.13 “控制台”视图的标题栏和工具栏

☒ 透视图工具栏

透视图工具栏主要包括切换已经打开的不同透视图的缩略按钮以及打开其他视图的按钮。在相应的工具按钮上单击鼠标右键会弹出透视图的管理菜单，实现透视图的定制、关闭、复位、布局位置、是否显示文本等操作，如图2.14所示。

☒ 快速视图工具栏

快速视图工具栏包含了一组快速视图的图标按钮，该工具栏中的视图会以图标按钮形式隐藏，单击指定工具按钮时会显示指定的视图，该视图失去焦点时会自动隐藏到快速工具栏。默认情况下，快速视图工具栏出现在工作台的左下角，如图2.15所示。



图2.14 透视图工具栏



图2.15 快速视图工具栏

2.1.9 “包资源管理器”视图



视频讲解：光盘\TM\lx\2\“包资源管理器”视图.exe

该视图用于浏览项目结构中的Java元素，包括包、类、类库的引用等，但最主要的用途还是操作项目中的源代码文件。“包资源管理器”视图的界面如图2.16所示。



图2.16 “包资源管理器”视图

2.1.10 “控制台”视图



视频讲解：光盘\TM\lx\2\“控制台”视图.exe

“控制台”视图用于显示程序运行的输出结果和异常信息（Runtime Exception）。在学习Swing程序设计之前，必须使用控制台实现与程序的交互，例如，为方便调试某个方法，该视图在方法执行前后会分别输出“方法开始”和“方法结束”信息。“控制台”视图的界面如图2.17所示。



图2.17 “控制台”视图

2.2 使用Eclipse

现在读者对Eclipse工具已经有大体的认识了，本节将介绍如何使用Eclipse完成HelloJava程序的编写和运行。（实例位置：光盘\TM\sl\Hello.java）

2.2.1 创建Java项目



视频讲解：光盘\TM\lx\2\创建Java项目.exe

在Eclipse中编写程序，必须先创建项目。Eclipse中有很多种项目，其中Java项目用于管理和编写Java程序。创建该项目的步骤如下：

（1）选择“文件” / “新建” / “ 项目”命令，打开“新建项目”对话框，该对话框包含创建项目的向导，在向导中选择“ Java项目”节点，单击“下一步”按钮。

（2）弹出“新建Java项目”对话框，在“项目名”文本框中输入“HelloJava”，在“项目布局”栏中选中“为源文件和类文件创建单独的文件夹”单选按钮，如图2.18所示，然后单击“完成”按钮，完成项目的创建。

2.2.2 创建Java类文件



视频讲解：光盘\TM\lx\2\创建Java类文件.exe

创建Java类文件时，会自动打开Java编辑器。创建Java类文件可以通过“新建Java类”向导来完成。在Eclipse菜单栏中选择“文件” / “新建” / “类”命令，将打开“新建Java类”向导对话框，如图2.19所示。

使用该向导对话框创建Java类的步骤如下：

（1）在“源文件夹”文本框中输入项目源程序文件夹的位置。通常向导会自动填写该文本框，没有特殊情况，不需要修改。

（2）在“包”文本框中输入类文件的包名，这里暂时默认为空，不输入任何信息，这样就会使用Java工程的默认包。

（3）在“名称”文本框中输入新建类的名称，如HelloJava。

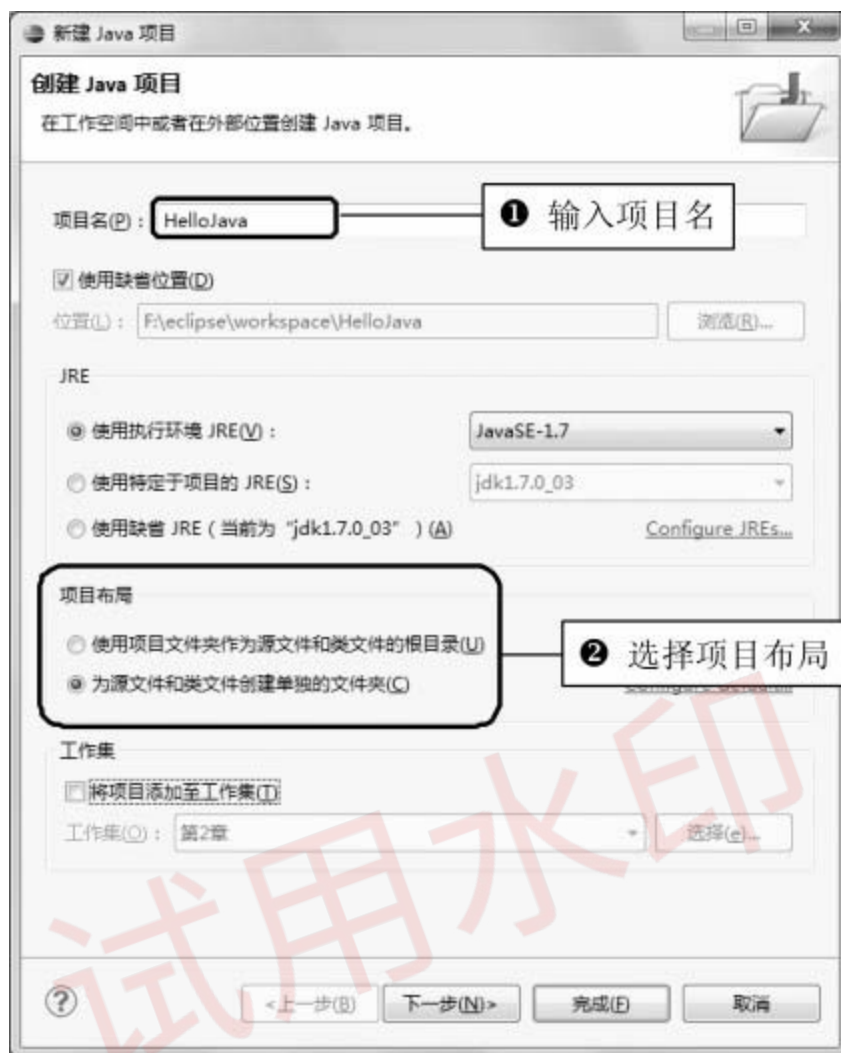


图2.18 “新建Java项目”对话框



图2.19 “新建Java类”向导对话框



注意

虽然HelloJava类名与Java项目同名，但是它们分别代表类文件和Java项目，读者需要注意区分它们的含义。鼠标右键并在弹出的快捷菜单中选择“打开方式”/“Java编辑器”命令。Java编辑器的界面如图2.20所示。

(4) 选中public static void main(String[] args)复选框，向导在创建类文件时，会自动为该类添加main()方法，使该类成为可以运行的主类。

2.2.3 使用编辑器编写程序代码



视频讲解：光盘\TM\lx\2\使用编辑器编写程序代码.exe

编辑器总是位于Eclipse工作台的中间区域，该区域可以重叠放置多个编辑器。编辑器的类型可以不同，但它们的主要功能都是完成Java程序、XML配置等代码编写或可视化设计工作。本节将介绍如何使用Java编辑器和其代码辅助功能快速编写Java程序。

1. 打开Java编辑器

在使用向导创建Java类文件之后，会自动打开Java编辑器编辑新创建的Java类文件。除此之外，打开Java编辑器最常用的方法是在“包资源管理器”视图中双击Java源文件或在Java源文件处单击

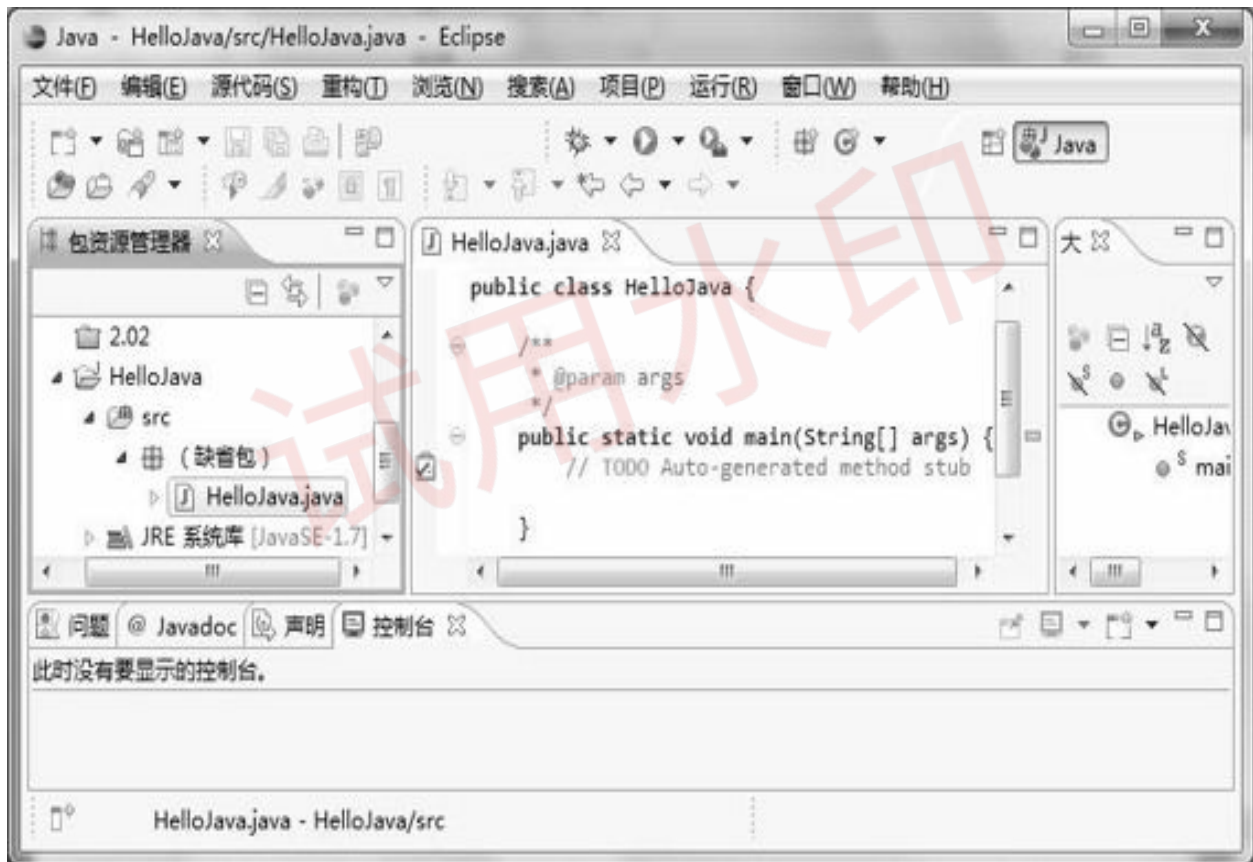


图2.20 Java编辑器界面

从图2.20中可以看到，Java编辑器以不同的样式和颜色突出显示Java语法。这些突出显示的语法包括以下几个方面：

- ☒ 程序代码注释。
- ☒ Javadoc注释。
- ☒ Java关键字。



技巧

在Java编辑器左侧单击鼠标右键，在弹出的快捷菜单中选择“显示行号”命令，可以开启Java编辑器显示行号的功能。

2. 编写Java代码

Eclipse的强大之处并不在于编辑器能突出显示Java语法，而在于它强大的代码辅助功能。在编写Java程序代码时，可以使用Ctrl+Alt+/快捷键自动补全Java关键字，也可以使用Alt+/快捷键启动Eclipse代码辅助菜单。

在使用向导创建HelloJava类之后，向导会自动构建HelloJava类结构的部分代码，并建立main()方法，程序开发人员需要做的就是将代码补全，为程序添加相应的业务逻辑。本程序的完整代码如图2.21所示。



技巧

在Eclipse安装后，Java编辑器文本字体为Consolas 10。采用这个字体时，中文显示比较小，不方便查看。这时，可以选择主菜单上的“窗口”/“首选项”命令，打开“首选项”对话框，在左侧的列表中选择“常规”/“外观”/“颜色和字体”节点，在右侧选择Java/“Java编辑器文本字体”节点，并单击“编辑”按钮，在弹出的对话框中选择Courier New字体，单击“确定”按钮，返回到“首选项”对话框中，单击“应用”按钮即可。另外，“调试”/“控制台字体”节点也需要进行以上修改。

在HelloJava程序代码中，第2、4、5、6行是由向导创建的，完成这个程序只要编写第3行和第8行代码即可。

首先来看一下第3行代码。它包括private、static、String 3个关键字，这3个关键字如果在记事本程序中手动输入可能不会花多长时间，但是无法避免出现输入错误的情况，如将private关键字输入为“privat”，缺少了字母“e”，这个错误可能在程序编译时才会被发现。如果是名称更长、更复杂的关键字，就更容易出现错误。而在Eclipse的Java编辑器中可以输入关键字的部分字母，然后使用Ctrl+Alt+/快捷键自动补全Java关键字，如图2.22所示。


```

1
2 public class HelloJava {
3     private static String say = "我要学会你。";
4     /**
5      * @param args
6      */
7     public static void main(String[] args) {
8         System.out.println("你好 Java " + say);
9     }
10 }
11 }

```

图2.21 HelloJava程序代码

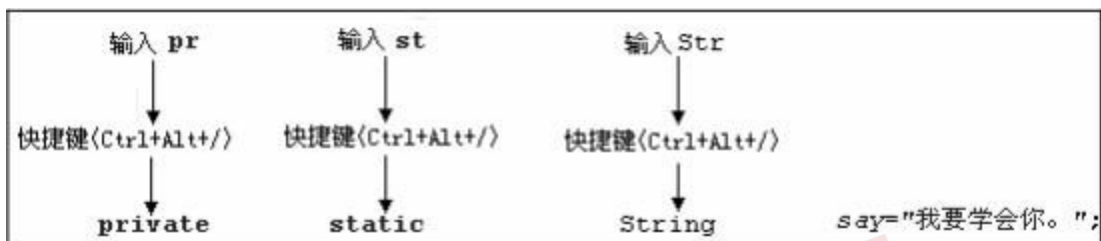


图2.22 使用快捷键补全关键字

其次是第8行的程序代码。它使用`System.out.println()`方法输出文字信息到控制台，这是程序开发时最常使用的方法之一。当输入“.”操作符时，编辑器会自动弹出代码辅助菜单，也可以在输入部分文字之后使用`Alt+/`快捷键调出代码辅助菜单，完成关键语法的输入，如图2.23所示。



注意

`System.out.println()`方法在Java编辑器中可以通过输入“syso”和按`Alt+/`快捷键完成快速输入。

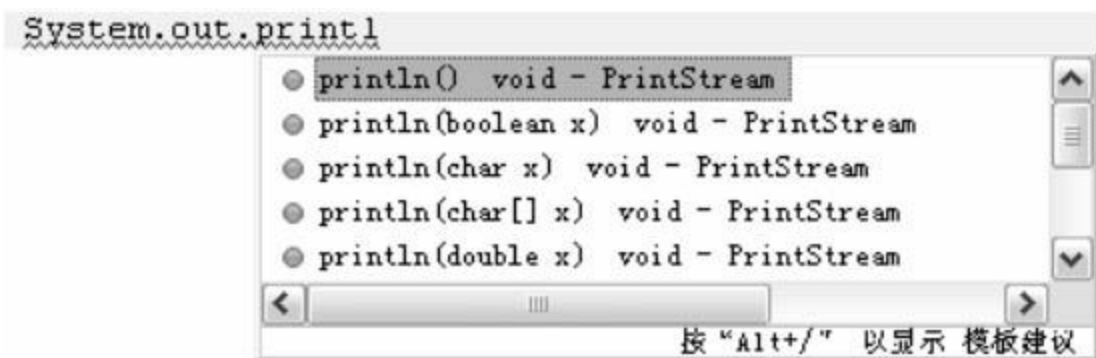


图2.23 代码辅助菜单



技巧

将光标移动到Java编辑器的错误代码位置，按Ctrl+1快捷键可以激活“代码修正”菜单，进而选择菜单中合适的修正方法。

2.2.4 运行Java程序



视频讲解：光盘\TM\lx\2\运行Java程序.exe

HelloJava类包含main()主方法，它是一个可以运行的主类。例如，在Eclipse中运行HelloJava程序，可以在“包资源管理器”视图中HelloJava文件处单击鼠标右键，在弹出的菜单中选择“运行方式” / “Java应用程序”命令。程序运行结果如图2.24所示。



图2.24 HelloJava程序在控制台的输出结果

2.3 程序调试



视频讲解：光盘\TM\lx\2\程序调试.exe

读者在程序开发过程中会不断体会到程序调试的重要性。为验证Java单元的运行状况，以往会在某个方法调用的开始和结束位置分别使用System.out.println()方法输出状态信息，并根据这些信息判断程序执行状况，但这种方法比较原始，而且经常导致程序代码混乱（导出的都是System.out.println()方法）。

本节将简单介绍Eclipse内置的Java调试器的使用方法，使用该调试器可以进行设置程序的断点，实现程序单步执行，在调试过程中查看变量和表达式的值等调试操作，这样可以避免在程序中编写大量的System.out.println()方法输出调试信息。

使用Eclipse的Java调试器需要设置程序断点，然后使用单步调试分别执行程序代码的每一行。

1. 断点

设置断点是程序调试中必不可少的手段，Java调试器每次遇到程序断点时都会将当前线程挂起，即暂停当前程序的运行。

可以在Java编辑器中显示代码行号的位置双击添加或删除当前行的断点，或者在当前行号的位置单击鼠标右键，在弹出的快捷菜单中选择“切换断点”命令实现断点的添加与删除，如图2.25所示。

2. 以调试方式运行Java程序

要在Eclipse中调试HelloJava程序，可以在“包资源管理器”视图中HelloJava文件处单击鼠标右键，在弹出的快捷菜单中选择“调试方式” / “Java应用程序”命令。在图2.25中第8行代码设置了断点，调试器将在该断点处挂起当前线程，使程序暂停，如图2.26所示。



图2.25 Java编辑器中的断点

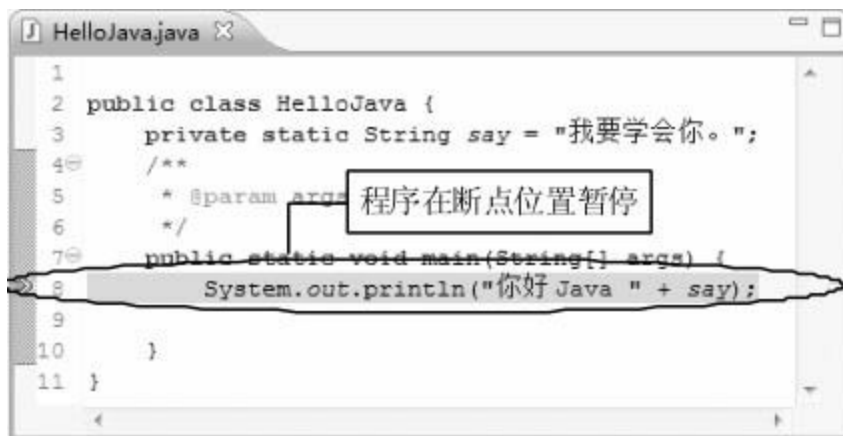


图2.26 程序执行到断点后暂停

3. 程序调试

程序执行到断点被暂停后，可以通过“调试”视图工具栏上的按钮执行相应的调试操作，如运行、停止等。“调试”视图如图2.27所示。

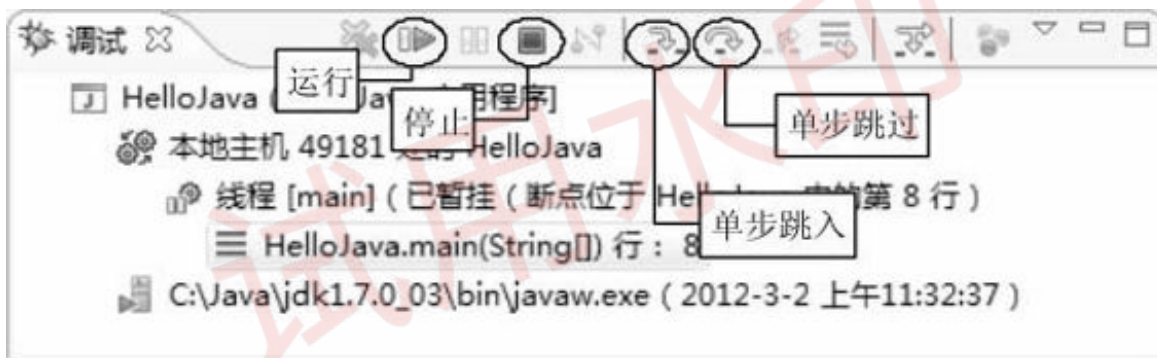


图2.27 “调试”视图

☒ 单步跳过

在“调试”视图的工具栏中单击  按钮或按F6键，将执行单步跳过操作，即运行单独的一行程序代码，但是不进入调用方法的内部，然后跳到下一个可执行点并暂停线程。



说明

不停地执行单步跳过操作，会每次执行一行程序代码，直到程序结束或等待用户操作。

☒ 单步跳入

在“调试”视图的工具栏中单击  按钮或按F5键，执行该操作将跳入调用方法或对象的内部单步执行程序并暂停线程。

2.4 小结

本章主要介绍了Eclipse开发工具，包括它的配置与启动、菜单栏、工具栏，以及常用的“包资源管理器”和“控制台”视图；另外还介绍了Eclipse的使用方法，包括创建Java项目、创建Java类、编写程序代码和运行程序；最后还讲解了Java程序调试器的简单使用方法。

通过对本章的学习，读者应该能初步认识并掌握Eclipse开发工具的使用。在学习本书内容时，应灵活运用Eclipse的代码辅助功能（使用Ctrl+Alt+/, Alt+/和Ctrl+1快捷键）快速编写程序代码。

2.5 实践与练习

1. 尝试在Eclipse中创建项目并编写程序，实现如图2.28所示的程序输出结果。（答案位置：光盘\TM\sl\2.01）

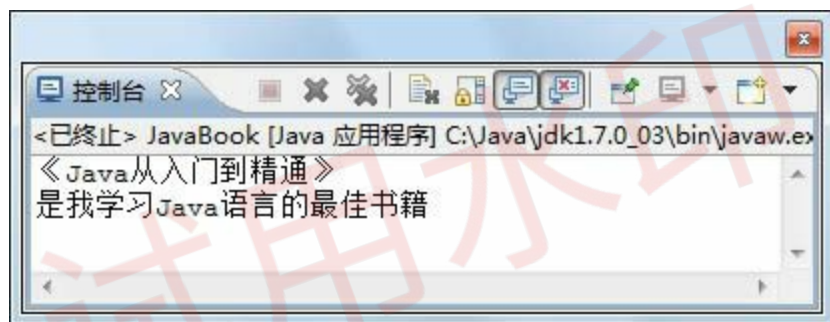


图2.28 程序输出结果

2. 尝试编写Java程序，使程序分别输出两个整数的加、减、乘、除运算结果。（答案位置：光盘\TM\sl\2.02）

第3章

Java语言基础

( 视频讲解：37分钟)

很多人认为学习Java语言之前必须要学习C++语言，其实并非如此，产生这种错误的认识是因为很多人在学习Java语言之前都学过C++语言。事实上，Java语言比C++语言更容易掌握。要掌握并熟练应用Java语言，就需要对Java语言的基础进行充分的了解。本章对Java语言基础进行了比较详细的介绍，对于初学者来说，应该对本章的各个小节进行仔细的阅读、思考，这样才能达到事半功倍的效果。

通过阅读本章，您可以：

- ▶▶ 了解Java 主类结构
- ▶▶ 了解Java 语言中的基本数据类型
- ▶▶ 理解Java 语言中的常量与变量
- ▶▶ 掌握Java 语言运算符的使用
- ▶▶ 理解Java 语言数据类型的转换
- ▶▶ 了解Java 语言中的代码注释与编码规范

3.1 Java主类结构



视频讲解：光盘\TM\lx\3\Java主类结构.exe

Java语言是面向对象的程序设计语言，Java程序的基本组成单元是类，类体中又包括属性与方法两部分（本书将在以下章节中逐一介绍）。每一个应用程序都必须包含一个main()方法，含有main()方法的类称为主类。下面通过程序来介绍Java主类结构。

【例3.1】在Eclipse下依次创建项目item、包Number和类Frist。在类体中输入以下代码，实现在控制台上输出“你好 Java”。（实例位置：光盘\TM\sl\3.01）

```
package Number;
public class Frist {
    static String s1 = "你好";
    public static void main(String[] args) {
        String s2 = "Java";
        System.out.println(s1);
        System.out.println(s2);
    }
}
```

运行结果如图3.1所示。

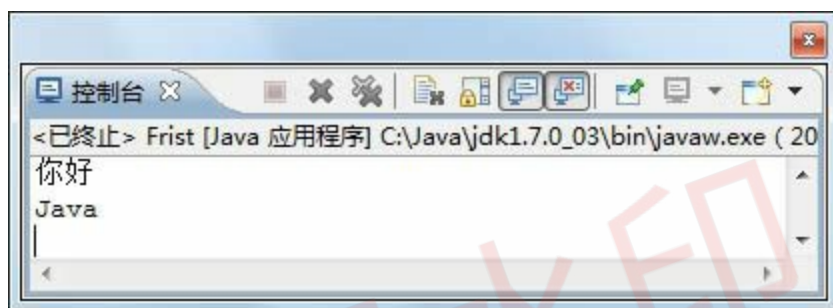


图3.1 在控制台上输出字符串



注意

代码中的所有标点符号都是英文字符，不要在中文输入法状态下输入标点符号，如双引号和分号，否则会导致编译错误。

文件名必须和类名Frist同名，即Frist.java。还要注意大小写，Java是区分大小写的。

3.1.1 包声明

一个Java应用程序是由若干个类组成的。在例3.1中就是一个类名为Frist的类，语句package Number为声明该类所在的包，package为包的关键字（关于包的详细介绍可参见第11章）。

3.1.2 声明成员变量和局部变量

通常将类的属性称之为类的全局变量（成员变量），将方法中的属性称之为局部变量。全局变量声明在类体中，局部变量声明在方法体

中。全局变量和局部变量都有各自的应用范围。在例3.1中s1是成员变量，s2是局部变量。

3.1.3 编写主方法

main()方法是类体中的主方法。该方法从“{”号开始，至“}”号结束。public、static和void分别是main()方法的权限修饰符、静态修饰符和返回值修饰符，Java程序中的main()方法必须声明为public static void。String[] args是一个字符串类型的数组，它是main()方法的参数（以后章节中将作详细的介绍）。main()方法是程序开始执行的位置。

3.1.4 导入API类库

在Java语言中可以通过import关键字导入相关的类。在JDK的API中（应用程序接口）提供了130多个包，如java.awt、java.io等。可以通过JDK的API文档来查看这些类，其中主要包括类的继承结构、类的应用、成员变量表、构造方法表等，并对每个变量的使用目的作了详细的描述，API文档是程序开发人员不可或缺的工具。



注意

Java语言是严格区分大小写的。例如，不能将关键字class等同于Class。

3.2 基本数据类型

在Java中有8种基本数据类型来存储数值、字符和布尔值，如图3.2所示。



图3.2 Java基本数据类型

3.2.1 整数类型



视频讲解：光盘\TM\lx\3\整数类型.exe

整数类型用来存储整数数值，即没有小数部分的数值。可以是正数，也可以是负数。整型数据在Java程序中有3种表示形式，分别为十进制、八进制和十六进制。

☒ 十进制：十进制的表现形式大家都很熟悉，如120、0、-127。



注意

不能以0作为十进制数的开头（0除外）。

☒ 八进制：如0123（转换成十进制数为83）、-0123（转换成十进制数为-83）。



注意

八进制必须以0开头。

☒ 十六进制：如0x25（转换成十进制数为37）、0Xb01e（转换成十进制数为45086）。



注意

十六进制必须以0X或0x开头。

整型数据根据它所占内存大小的不同，可分为byte、short、int和long 4种类型。它们具有不同的取值范围，如表3.1所示。

表3.1 整型数据类型

数据类型	内存空间 (8 位等于 1 字节)	取值范围
byte	8 位	-128~127
short	16 位	-32768~32767
int	32 位	-2147483648~2147483647
long	64 位	-9223372036854775808~9223372036854775807

下面以int型变量为例讲解整型变量的定义。

【例3.2】定义int型变量，实例代码如下：

```
int x;           //定义 int 型变量 x
int x,y;         //定义 int 型变量 x、y
int x = 450,y = -462; //定义 int 型变量 x、y 并赋给初值
```

在定义以上4种类型变量时，要注意变量的取值范围，超出相应范围就会出错。对于long型值，若赋给的值大于int型的最大值或小于int型的最小值，则需要在数字后加L或l，表示该数值为长整数，如long num = 2147483650L。

【例3.3】在项目中创建类Number，在主方法中创建不同数值型变量，并将这些变量相加，将和输出。（实例位置：光盘\TM\sl\3.02）

```
public class Number {           //创建类
    public static void main(String[] args) {           //主方法
        byte mybyte = 124;           //声明 byte 型变量并赋值
        short myshort = 32564;       //声明 short 型变量并赋值
        int myint = 45784612;        //声明 int 型变量并赋值
        long mylong = 46789451;      //声明 long 型变量并赋值
        long result = mybyte + myshort + myint + mylong; //获得各数相加后的结果
        System.out.println("结果为: " + result);       //将以上变量相加的结果输出
    }
}
```

程序运行结果如图3.3所示。

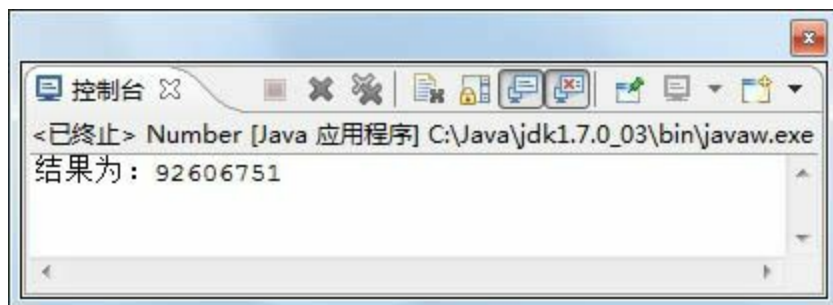


图3.3 不同数值型变量相加的和

3.2.2 浮点类型



视频讲解：光盘\TM\lx\3\浮点类型.exe

浮点类型表示有小数部分的数字。Java语言中浮点类型分为单精度浮点类型（float）和双精度浮点类型（double），它们具有不同的取值范围，如表3.2所示。

表3.2 浮点型数据类型

数据类型	内存空间（8位等于1字节）	取值范围
float	32位	1.4E-45~3.4028235E38
double	64位	4.9E-324~1.7976931348623157E308

在默认情况下小数都被看作double型，若使用float型小数，则需要小数后面添加F或f。可以使用后缀d或D来明确表明这是一个double类型数据，不加“d”不会出错，但声明float型变量时如果不加“f”，系统会认为变量是double类型而出错。下面举例介绍声明浮点类型变量的方法。

【例3.4】定义浮点类型变量，实例代码如下：

```
float f1 = 13.23f;
double d1 = 4562.12d;
double d2 = 45678.1564;
```

3.2.3 字符类型



视频讲解：光盘\TM\lx\3\字符类型.exe

1. char型

字符类型（**char**）用于存储单个字符，占用16位（两个字节）的内存空间。在定义字符型变量时，要以单引号表示，如's'表示一个字符。而's'则表示一个字符串，虽然只有一个字符，但由于使用双引号，它仍然表示字符串，而不是字符。

使用**char**关键字可定义字符变量，下面举例说明。【例3.5】声明字符型变量，实例代码如下：

```
char x = 'a';
```

由于字符a在unicode表中的排序位置是97，因此允许将上面的语句写成：

```
char x = 97;
```

同C和C++语言一样，Java语言也可以把字符作为整数对待。由于unicode编码采用无符号编码，可以存储65536个字符（0x0000～0xffff），所以Java中的字符几乎可以处理所有国家的语言文字。若想得到一个0～65536之间的数所代表的unicode表中相应位置上的字符，也必须使用**char**型显式转换。

【例3.6】在项目中创建类Gess，编写如下代码，实现将unicode表中某些位置上的字符以及一些字符在unicode表中的位置在控制台上输出。（实例位置：光盘\TM\sl\3.03）

```
public class Gess { //定义类
    public static void main(String[] args) { //主方法
        char word = 'd', word2 = '@'; //定义 char 型变量
        int p = 23045, p2 = 45213; //定义 int 型变量
        System.out.println("d 在 unicode 表中的顺序位置是: " + (int) word);
        System.out.println("@在 unicode 表中的顺序位置是: " + (int) word2);
        System.out.println("unicode 表中的第 23045 位是: " + (char) p);
        System.out.println("unicode 表中的第 45213 位是: " + (char) p2);
    }
}
```

运行结果如图3.4所示。

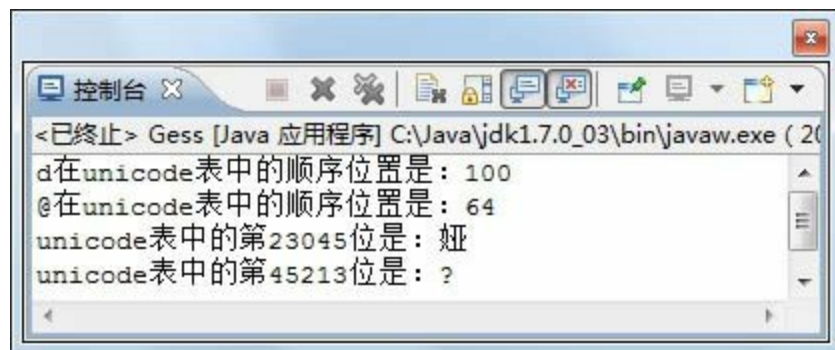


图3.4 例3.6的运行结果

2. 转义字符

转义字符是一种特殊的字符变量，它以反斜线“\”开头，后跟一个或多个字符。转义字符具有特定的含义，不同于字符原有的意义，故称“转义”。例如，printf函数的格式串中用到的“\n”就是一个转义字符，意思是“回车换行”。Java中的转义字符如表3.3所示。

表3.3 转义字符

转 义 字 符	含 义
\ddd	1~3 位八进制数据所表示的字符，如\456
\dxxxx	4 位十六进制所表示的字符，如\0052
\'	单引号字符
\\	反斜杠字符
\t	垂直制表符，将光标移到下一个制表符的位置
\r	回车
\n	换行
\b	退格
\f	换页

将转义字符赋值给字符变量时，与字符常量值一样需要使用单引号。

【例3.7】使用转义字符，实例代码如下：

```
char c1 = '\\';           //将转义字符 '\\' 赋值给变量 c1
char char1 = '\u2605';    //将转义字符 '\u2605' 赋值给变量 char1
System.out.println(c1);   //输出结果\
System.out.println(char1); //输出结果★
```

3.2.4 布尔类型



视频讲解：光盘\TM\lx\3\布尔类型.exe

布尔类型又称逻辑类型，通过关键字**boolean**来定义布尔类型变量，只有**true**和**false**两个值，分别代表布尔逻辑中的“真”和“假”。布尔值不能与整数类型进行转换。布尔类型通常被用在流程控制中作为判断条件。

【例3.8】声明boolean型变量，实例代码如下：

```
boolean b;           //定义布尔型变量 b
boolean b1,b2;       //定义布尔型变量 b1、b2
boolean b = true;    //定义布尔型变量 b，并赋给初值 true
```

3.3 变量与常量

在程序执行过程中，其值不能被改变的量称为常量，其值能被改变的量称为变量。变量与常量的命名都必须使用合法的标识符。本节将向读者介绍标识符与关键字、变量与常量的命名。

3.3.1 标识符和关键字



视频讲解：光盘\TM\lx\3\标识符和关键字.exe

1. 标识符

标识符可以简单地理解为一个名字，用来标识类名、变量名、方法名、数组名、文件名的有效字符序列。

Java语言规定标识符由任意顺序的字母、下划线（_）、美元符号（\$）和数字组成，并且第一个字符不能是数字。标识符不能是Java中的保留关键字。

下面是合法标识符：

```
name
user_age
$page
```

下面是非法标识符：

```
4word
String
User name
```

在Java语言中标识符中的字母是严格区分大小写的，如good和Good是不同的两个标识符。Java语言使用unicode标准字符集，最多可以标识65535个字符，因此，Java语言中的字母不仅包括通常的拉丁文字a、b、c等，还包括汉字、日文以及其他许多语言中的文字。

2. 关键字

关键字是Java语言中已经被赋予特定意义的一些单词，不可以把这些字作为标识符来使用。3.2节介绍的数据类型中提到的int、boolean等都是关键字。Java语言中的关键字如表3.4所示。

表3.4 Java关键字

int	public	this	finally	boolean	abstract
continue	float	long	short	throw	throws
return	break	for	static	new	interface
if	goto	default	byte	do	case
strictfp	package	super	void	try	switch
else	catch	implements	private	final	class
extends	volatile	while	synchronized	instanceof	char
protecte	importd	transient	implements	dafaule	double

3.3.2 声明变量



视频讲解：光盘\TM\lx\3\声明变量.exe

变量的使用是程序设计中一个十分重要的环节。定义变量就是要告诉编译器（**compiler**）这个变量的数据类型，这样编译器才知道需要配置多少空间给它，以及它能存放什么样的数据。在程序运行过程中，空间内的值是变化的，这个内存空间就称为变量。为了便于操作，给这个空间取个名字，称为变量名。变量的命名必须是合法的标识符。内存空

间的值就是变量值。在声明变量时可以是没有赋值，也可以是直接赋给初值。

【例3.9】声明变量，实例代码如下：

```
int age;           //声明 int 型变量
char char1 = 'r';  //声明 char 型变量并赋值
```

编写以上程序代码，究竟会产生什么样的效果呢？要了解这个问题，就需要对变量的内存配置有一定的认识。用图解的方式将上例程序代码在内存中的状况表现出来，如图3.5所示。

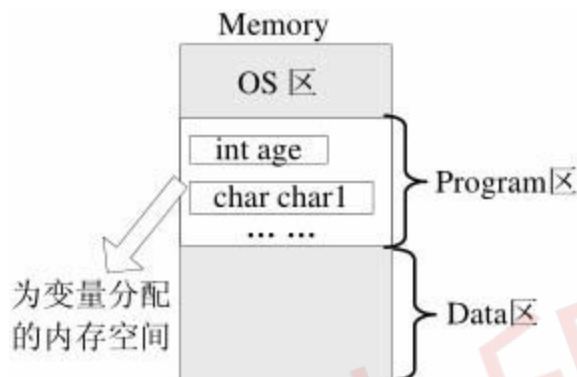


图3.5 变量占用的内存空间

由图3.5可知，系统的内存可大略分为3个区域，即系统区（OS）、程序区（Program）和数据区（Data）。当程序执行时，程序代码会加载到内存中的程序区，数据暂时存储在数据区中。假设上述两个变量定义在方法体中，则程序加载到程序区中。当执行此行程序代码时，会在数据区配置空间给出这两个变量。

对于变量的命名并不是任意的，应遵循以下几条规则：

- ☑ 变量名必须是一个有效的标识符。
- ☑ 变量名不可以使用Java中的关键字。
- ☑ 变量名不能重复。
- ☑ 应选择有意义的单词作为变量名。



说明

在Java语言中允许使用汉字或其他语言文字作为变量名，如“int年龄 = 21”，在程序运行时不会出现错误，但建议读者尽量不要使用这些语言文字作为变量名。

3.3.3 声明常量



视频讲解：光盘\TM\lx\3\声明常量.exe

在程序运行过程中一直不会改变的量称为常量（constant），通常也被称为“final变量”。常量在整个程序中只能被赋值一次。在为所有的对象共享值时，常量是非常有用的。

在Java语言中声明一个常量，除了要指定数据类型外，还需要通过final关键字进行限定。声明常量的标准语法如下：

```
final 数据类型 常量名称[=值]
```

常量名通常使用大写字母，但这并不是必须的。很多Java程序员使用大写字母表示常量，是为了清楚地表明正在使用常量。

【例3.10】声明常量，实例代码如下：

```
final double PI = 3.1415926D;    //声明 double 型常量 PI 并赋值
```

```
final boolean BOOL = true;      //声明 boolean 型常量 BOOL 并赋值
```

当定义的final变量属于“成员变量”时，必须在定义时就设定它的初值，否则将会产生编译错误。从下面的实例中可看出变量与常量的区别。

【例3.11】在项目中创建类Part，在类体中创建变量age与常量PI。在主方法中分别将变量与常量赋值，通过输出信息可测试变量与常量的有效范围。（实例位置：光盘\TM\sl\3.04）

```
public class Part {                //新建类 Part
    //声明常量 PI，此时如不对 PI 进行赋值，则会出现错误提示

    static final double PI = 3.14;

    static int age = 23;            //声明 int 型变量 age 并进行赋值

    public static void main(String[] args) { //主方法
        final int number;           //声明 int 型常量 number
        number = 1235;              //对常量进行赋值
        age = 22;                   //再次对变量进行赋值
        //number = 1236;             //错误的代码，因为 number 为常量，只能进行一次赋值
        System.out.println("常量 PI 的值为: " + PI);    //将 PI 的值输出
        System.out.println("赋值后 number 的值为: " + number); //将 number 的值输出
        System.out.println("int 型变量 age 的值为: " + age); //将 age 的值输出
    }
}
```

运行结果如图3.6所示。

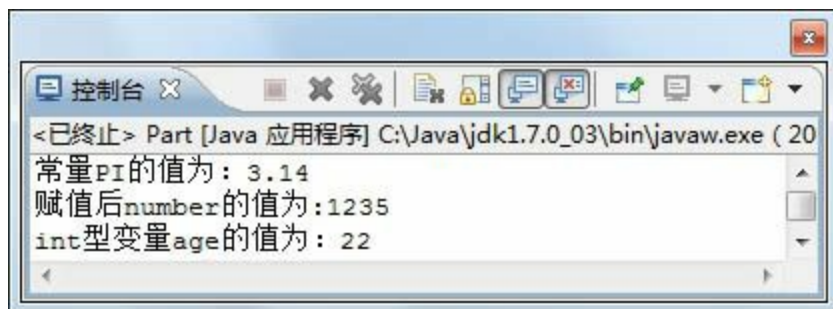


图3.6 例3.11的运行结果

3.3.4 变量的有效范围



视频讲解：光盘\TM\lx\3\变量的有效范围.exe

由于变量被定义出来后只是暂存在内存中，等到程序执行到某一点，该变量会被释放掉，也就是说变量有它的生命周期。因此，变量的有效范围是指程序代码能够访问该变量的区域，若超出该区域，则在编译时会出现错误。在程序中，一般会根据变量的“有效范围”将变量分为“成员变量”和“局部变量”。

1. 成员变量

在类体中所定义的变量被称为成员变量，成员变量在整个类中都有效。类的成员变量又可分为两种，即静态变量和实例变量。

【例3.12】声明静态变量和实例变量，实例代码如下：

```
class var{  
    int x = 45;  
    static int y = 90  
}
```

其中，x为实例变量，y为静态变量（也称类变量）。如果在成员变量的类型前面加上关键字static，这样的成员变量称为静态变量。静态变量的有效范围可以跨类，甚至可达到整个应用程序之内。对于静态变量，除了能在定义它的类内存取，还能直接以“类名.静态变量”的方式在其他类内使用。

2. 局部变量

在类的方法体中定义的变量（方法内部定义，“{”与“}”之间的代码中声明的变量）称为局部变量。局部变量只在当前代码块中有效。

在类的方法中声明的变量，包括方法的参数，都属于局部变量。局部变量只在当前定义的方法内有效，不能用于类的其他方法中。局部变量的生命周期取决于方法，当方法被调用时，Java虚拟机为方法中的局部变量分配内存空间，当该方法的调用结束后，则会释放方法中局部变量占用的内存空间，局部变量也将会销毁。

局部变量可与成员变量的名字相同，此时成员变量将被隐藏，即这个成员变量在此方法中暂时失效。

变量的有效范围如图3.7所示。

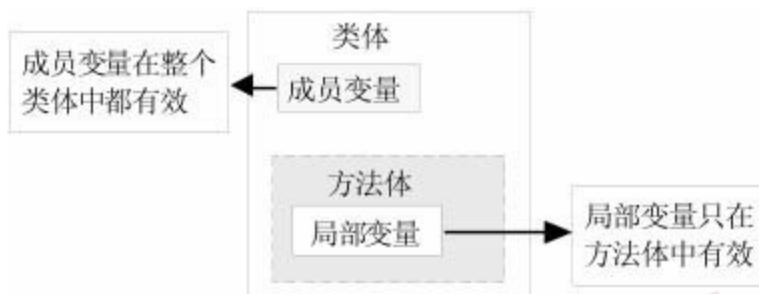


图3.7 变量的有效范围

【例3.13】在项目中创建类Val，分别定义名称相同的局部变量与成员变量，当名称相同时成员变量将被隐藏。（实例位置：光盘\TM\sl\3.05）

```
public class Val { //新建类
    static int times = 3; //定义成员变量 times
    public static void main(String[] args) { //主方法
        int times = 4; //定义局部变量 times
        System.out.println("times 的值为: " + times); //将 times 的值输出
    }
}
```

运行结果如图3.8所示。

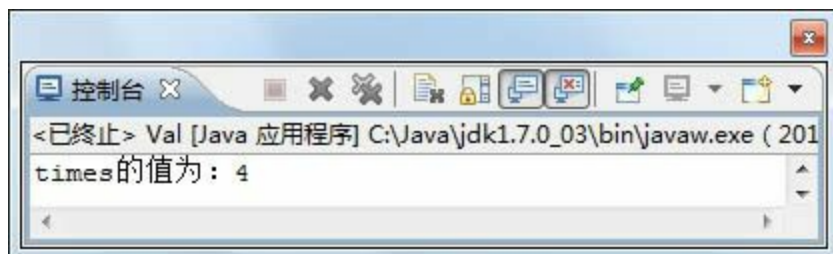


图3.8 例3.13的运行结果

3.4 运算符

运算符是一些特殊的符号，主要用于数学函数、一些类型的赋值语句和逻辑比较方面。Java中提供了丰富的运算符，如赋值运算符、算术运算符、比较运算符等。本节将向读者介绍这些运算符。

3.4.1 赋值运算符



视频讲解：光盘\TM\lx\3\赋值运算符.exe

赋值运算符以符号“=”表示，它是一个二元运算符（对两个操作数作处理），其功能是将右方操作数所含的值赋给左方的操作数。例如：

```
int a = 100;
```

该表达式是将100赋值给变量a。左方的操作数必须是一个变量，而右边的操作数则可以是任何表达式，包括变量（如a、number）、常量（如123、‘book’）、有效的表达式（如45*12）。

【例3.14】使用赋值运算符为变量赋值，实例代码如下：

```
int a = 10;           //声明 int 型变量 a
int b = 5;            //声明 int 型变量 b
int c = a+b;          //将变量 a 与 b 运算后的结果赋值给 c
```

遵循赋值运算符的运算规则，可知系统将先计算a+b的值，结果为15，然后将15赋值给变量c，因此“c=15”。

由于赋值运算符“=”处理时会先取得右方表达式处理后的结果，因此一个表达式中若含有两个以上的“=”运算符，会从最右方的“=”开始处理。

【例3.15】在项目中创建类Eval，在主方法中定义变量，使用赋值运算符为变量赋值。（实例位置：光盘\TM\sl\3.06）

```
public class Eval {           //创建类
    public static void main(String[] args) { //主方法
        int a, b, c;           //声明 int 行变量 a、b、c
        a = 15;                //将 15 赋值给变量 a
        c = b = a + 4;          //将 a 与 4 的和赋值给变量 b，然后再赋值给变量 c
        System.out.println("c 值为: " + c); //将变量 c 的值输出
        System.out.println("b 值为: " + b); //将变量 b 的值输出
    }
}
```

运行结果如图3.9所示。

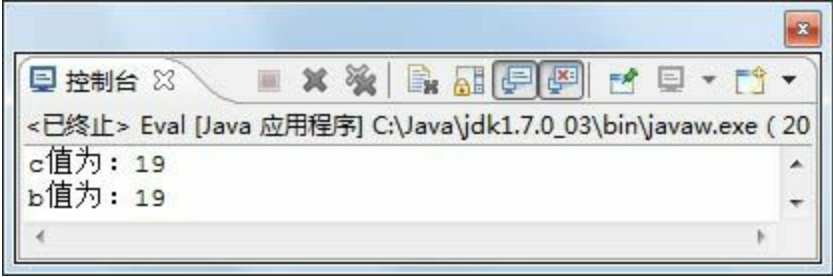


图3.9 例3.15的运行结果



说明

在Java中可以把赋值运算符连在一起使用。如：
`x = y = z = 5;`
在这个语句中，变量x、y、z都得到同样的值5。但在程序开发中不建议使用这种赋值语句。

3.4.2 算术运算符



视频讲解：光盘\TM\lx\3\算术运算符.exe

Java中的算术运算符主要有+（加）、-（减）、*（乘）、/（除）、%（求余），它们都是二元运算符。Java中算术运算符的功能及使用方式如表3.5所示。

表3.5 Java算术运算符

运 算 符	说 明	实 例	结 果
+	加	12.45f+15	27.45
-	减	4.56-0.16	4.4
*	乘	5L*12.45f	62.25
/	除	7/2	3
%	取余数	12%10	2

其中，“+”和“-”运算符还可以作为数据的正负符号，如+5、-7。



注意

在进行除法运算时，0不能做除数。例如，对于语句“`inta = 5/0;`”，系统会抛出

ArithmeticException异常。

下面通过一个小程序来介绍算术运算符的使用方法。

【例3.16】在项目中创建类Arith，在主方法中定义变量，使用算术运算符将变量的计算结果输出。（实例位置：光盘\TM\sl\3.07）

```
public class Arith {                                //创建类
    public static void main(String[] args) {        //主方法
        float number1 = 45.56f;                    //声明 float 型变量并赋值
        int number2 = 152;                          //声明 int 型变量并赋值
        System.out.println("和为: " + number1 + number2); //将变量相加之和输出
        System.out.println("差为: " + (number2-number1)); //将变量相减之差输出
        System.out.println("积为: " + number1 * number2); //将变量相乘的积输出
        System.out.println("商为: " + number1 / number2); //将变量相除的商输出
    }
}
```

运行结果如图3.10所示。

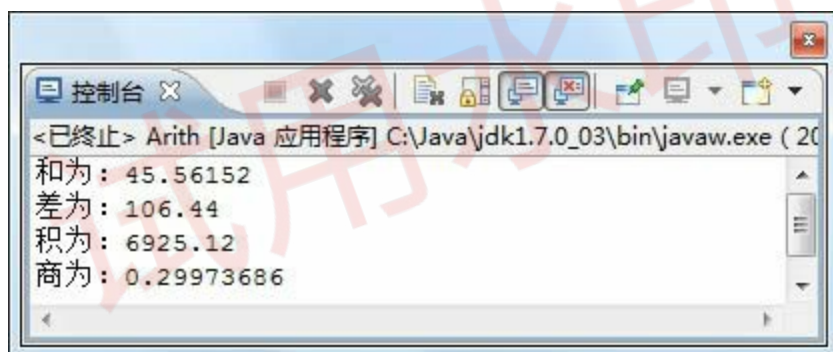


图3.10 使用算术运算符将变量的计算结果输出

3.4.3 自增和自减运算符



视频讲解：光盘\TM\lx\3\自增和自减运算符.exe

自增、自减运算符是单目运算符，可以放在操作元之前，也可以放在操作元之后。操作元必须是一个整型或浮点型变量。自增、自减运算符的作用是使变量的值增1或减1。放在操作元前面的自增、自减运算符，会先将变量的值加1（减1），然后再使该变量参与表达式的运算。放在操作元后面的自增、自减运算符，会先使变量参与表达式的运算，然后再将该变量加1（减1）。例如：

<code>++a(--a)</code>	//表示在使用变量 a 之前，先使 a 的值加（减）1
<code>a++(a--)</code>	//表示在使用变量 a 之后，使 a 的值加（减）1

粗略地分析，`++a`与`a++`的作用都相当于`a = a+1`。假设`a = 4`，则：

<code>b = ++a;</code>	//先将 a 的值加 1，然后赋给 b，此时 a 值为 5，b 值为 5
-----------------------	--------------------------------------

再看另一个语法，同样假设`a = 4`，则：

<code>b = a++;</code>	//先将 a 的值赋给 b，再将 a 的值变为 5，此时 a 值为 5，b 值为 4
-----------------------	--

3.4.4 比较运算符

 视频讲解：光盘\TM\lx\3\比较运算符.exe

比较运算符属于二元运算符，用于程序中的变量之间、变量和自变量之间以及其他类型的信息之间的比较。比较运算符的运算结果是boolean型。当运算符对应的关系成立时，运算结果为true，否则为false。所有比较运算符通常作为判断的依据用在条件语句中。比较运算符共有6个，如表3.6所示。

表3.6 比较运算符

运 算 符	作 用	举 例	操 作 数 据	结 果
>	比较左方是否大于右方	'a'>'b'	整型、浮点型、字符型	false
<	比较左方是否小于右方	156 < 456	整型、浮点型、字符型	false
==	比较左方是否等于右方	'c'=='c'	基本数据类型、引用型	true
>=	比较左方是否大于等于右方	479>=426	整型、浮点型、字符型	true
<=	比较左方是否小于等于右方	12.45<=45.5	整型、浮点型、字符型	true
!=	比较左方是否不等于右方	'y'!='t'	基本数据类型、引用型	true

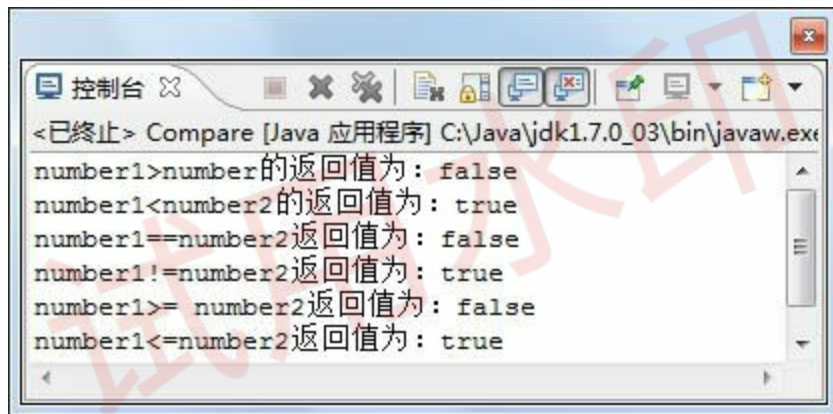
【例3.17】在项目中创建类Compare，在主方法中创建整型变量，使用比较运算符对变量进行比较运算，并将运算后的结果输出。（实例位置：光盘\TM\sl\3.08）

```

public class Compare {                                //创建类
    public static void main(String[] args) {
        int number1 = 4;                               //声明 int 型变量 number1
        int number2 = 5;                               //声明 int 型变量 number2
        /* 依次将变量 number1 与变量 number2 的比较结果输出 */
        System.out.println("number1>number 的返回值为: " +(number1 > number2));
        System.out.println("number1==number2 返回值为: " +(number1== number2));
        System.out.println("number1!=number2 返回值为: " +(number1 != number2));
        System.out.println("number1>= number2 返回值为: " +(number1 >= number2));
        System.out.println("number1<=number2 返回值为: " +(number1 <= number2));
    }
}

```

运行结果如图3.11所示。



```

<已终止> Compare [Java 应用程序] C:\Java\jdk1.7.0_03\bin\javaw.exe
number1>number的返回值为: false
number1<number2的返回值为: true
number1==number2返回值为: false
number1!=number2返回值为: true
number1>= number2返回值为: false
number1<=number2返回值为: true

```

图3.11 例3.17的运行结果

3.4.5 逻辑运算符



视频讲解：光盘\TM\lx\3\逻辑运算符.exe

返回类型为布尔值的表达式，如比较运算符，可以被组合在一起构成一个更复杂的表达式。这是通过逻辑运算符来实现的。逻辑运算符包括&（&&）（逻辑与）、||（逻辑或）、！（逻辑非）。逻辑运算符的操作元必须是boolean型数据。在逻辑运算符中，除了“！”是一元运算符之外，其他都是二元运算符。表3.7给出了逻辑运算符的用法和含义。

表3.7 逻辑运算符

运 算 符	含 义	用 法	结 合 方 向
&&、&	逻辑与	op1&&op2	左到右
	逻辑或	op1 op2	左到右
!	逻辑非	!op	右到左

结果为boolean型的变量或表达式可以通过逻辑运算符组合为逻辑表达式。

用逻辑运算符进行逻辑运算时，结果如表3.8所示。

表3.8 使用逻辑运算符进行逻辑运算

表达式 1	表达式 2	表达式 1&&表达式 2	表达式 1 表达式 2	!表达式 1
true	true	true	true	false
true	false	false	true	false
false	false	false	false	true
false	true	false	true	true

逻辑运算符“&&”与“&”都表示“逻辑与”，那么它们之间的区别在哪里呢？从表3.8可以看出，当两个表达式都为true时，“逻辑与”的结果才会是true。使用逻辑运算符“&”会判断两个表达式；而逻辑运算符“&&”则是针对boolean类型的类进行判断，当第一个表达式为false时则不去判断第二个表达式，直接输出结果从而节省计算机判断的次数。通常将这种在逻辑表达式中从左端的表达式可推断出整个表达式的值称为“短路”，而那些始终执行逻辑运算符两边的表达式称为“非短路”。“&&”属于“短路”运算符，而“&”则属于“非短路”运算符。

【例3.18】在项目中创建类Calculation，在主方法中创建整型变量，使用逻辑运算符对变量进行运算，并将运算结果输出。（实例位置：光盘\TM\sl\3.09）


```

public class Calculation {           //创建类
    public static void main(String[] args) {
        int a = 2;                   //声明 int 型变量 a
        int b = 5;                   //声明 int 型变量 b
                                     //声明 boolean 型变量，用于保存应用逻辑运算符“&&”后的返回值
        boolean result = ((a > b) && (a != b));
                                     //声明 boolean 型变量，用于保存应用逻辑运算符“||”后的返回值
        boolean result2 = ((a > b) || (a != b));
        System.out.println(result);  //将变量 result 输出
        System.out.println(result2); //将变量 result2 输出
    }
}

```

运行结果如图3.12所示。

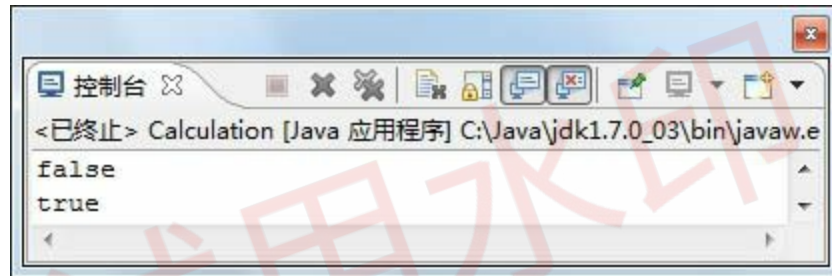


图3.12 例3.18的运行结果

3.4.6 位运算符



视频讲解：光盘\TM\3\位运算符.exe

位运算符除按位与和按位或运算符外，其他只能用于处理整数的操作数。位运算是完全针对位方面的操作。整型数据在内存中以二进制的形式表示，如int型变量7的二进制表示是00000000 00000000 00000000 00000111。

左边最高位是符号位，最高位是0表示正数，若为1则表示负数。负数采用补码表示，如-8的二进制表示为11111111 11111111 11111111 11111000。这样就可以对整型数据进行按位运算。

1. “按位与”运算

“按位与”运算的运算符为“&”，为双目运算符。“按位与”运算的算法则是：如果两个整型数据a、b对应位都是1，则结果位才是1，否则

为0。如果两个操作数的精度不同，则结果的精度与精度高的操作数相同，如图3.13所示。

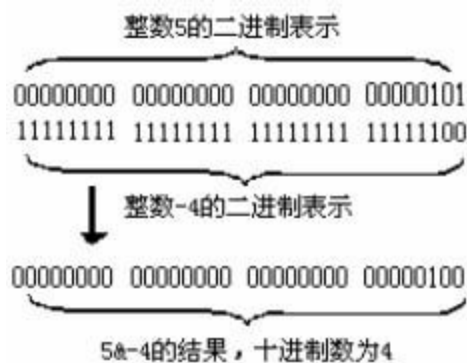


图3.13 5&-4的运算过程

2. “按位或”运算

“按位或”运算的运算符为“|”，为双目运算符。“按位或”运算的运算法则是：如果两个操作数对应位都是0，则结果位才是0，否则为1。如果两个操作数的精度不同，则结果的精度与精度高的操作数相同，如图3.14所示。

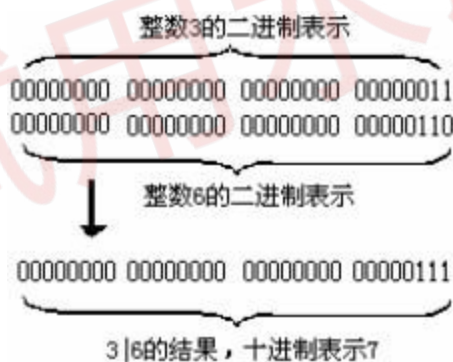


图3.14 3|6的运算过程

3. “按位取反”运算

“按位取反”运算也称“按位非”运算，运算符为“~”，为单目运算符。“按位取反”就是将操作数二进制中的1修改为0，0修改为1，如图3.15所示。

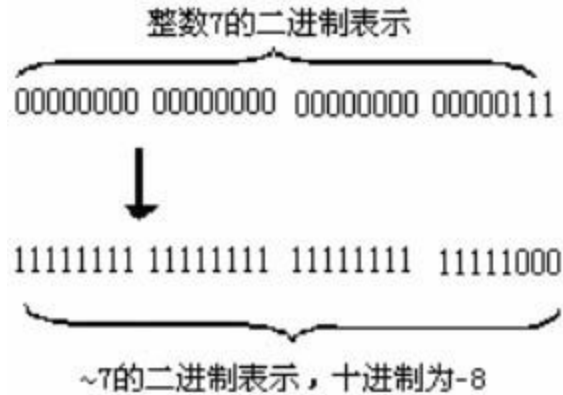


图3.15 ~7的运算过程

4. “按位异或”运算

“按位异或”运算的运算符是“^”，为双目运算符。“按位异或”运算的运算法则是：当两个操作数的二进制表示相同（同时为0或同时为1）时，结果为0，否则为1。若两个操作数的精度不同，则结果数的精度与精度高的操作数相同，如图3.16所示。

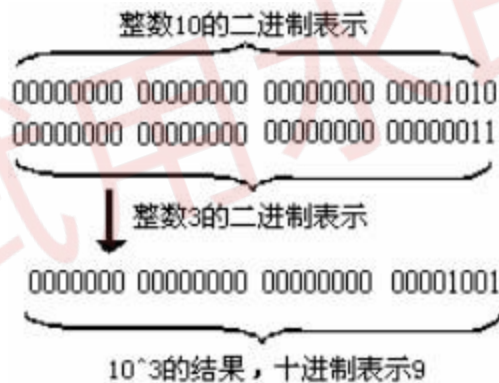


图3.16 10^3的运算过程

5. 移位操作

除了上述运算符之外，还可以对数据按二进制位进行移位操作。Java中的移位运算符有以下3种。

- ☒ <<: 左移。
- ☒ >>: 右移。
- ☒ >>>: 无符号右移。

左移就是将左边的操作数在内存中的二进制数据左移右边操作数指定的位数，左边移空的部分补0。右移则复杂一些。当使用“>>”符号时，如果最高位是0，左移空的位就填入0；如果最高位是1，右移空的

位就填入1，如图3.17所示。

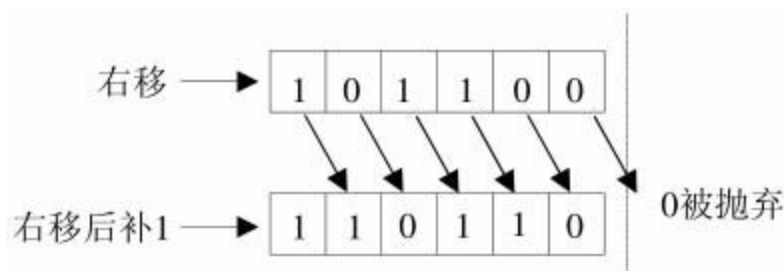


图3.17 右移

Java还提供了无符号右移“>>>”，无论最高位是0还是1，左侧被移空的高位都填入0。



注意

移位运算符适用的数据类型有byte、short、char、int和long。



技巧

移位可以实现整数除以或乘以 2^n 的效果。例如， $y << 2$ 与 $y * 4$ 的结果相同； $y >> 1$ 的结果与 $y / 2$ 的结果相同。总之，一个数左移 n 位，就是将这个数乘以 2^n ；一个数右移 n 位，就是将这个数除以 2^n 。

3.4.7 三元运算符



视频讲解：光盘\TM\lx\3\三元运算符.exe

三元运算符的使用格式为：

条件式?值 1:值 2

三元运算符的运算规则为：若条件式的值为true，则整个表达式取值1，否则取值2。例如：

```
boolean b = 20 < 45 ? true : false;
```

如上例所示，表达式“20<45”的运算结果返回真，那么boolean型变量b取值为true；相反，表达式“45<20”返回为假，则boolean型变量b取值false。

三元运算符等价于if...else语句。

【例3.19】等价于三元运算符的if...else语句，实例代码如下：（实例位置：光盘\TM\sl\3.10）

```
boolean a;           //声明 boolean 型变量
if(20<45)             //将 20<45 作为判断条件
    a = true;         //条件成立将 true 赋值给 a
else
    a = false;        //条件不成立将 false 赋值给 a
```

3.4.8 运算符优先级



视频讲解：光盘\TM\lx\3\运算符优先级.exe

Java中的表达式就是使用运算符连接起来的符合Java规则的式子。运算符的优先级决定了表达式中运算执行的先后顺序。通常优先级由高到低的顺序依次是：

- ☒ 增量和减量运算。
- ☒ 算术运算。
- ☒ 比较运算。
- ☒ 逻辑运算。
- ☒ 赋值运算。

如果两个运算有相同的优先级，那么左边的表达式要比右边的表达式先被处理。表3.9显示了在Java中众多运算符特定的优先级。

表3.9 运算符的优先级

优 先 级	描 述	运 算 符
1	括号	()
2	正负号	+, -
3	一元运算符	++, --, !
4	乘除	*, /, %
5	加减	+, -
6	移位运算	>>, >>>, <<
7	比较大小	<, >, >=, <=
8	比较是否相等	==, !=
9	按位与运算	&
10	按位异或运算	^
11	按位或运算	
12	逻辑与运算	&&
13	逻辑或运算	
14	三元运算符	? :
15	赋值运算符	=



技巧

在编写程序时尽量使用括号“()”运算符来限定运算次序，以免产生错误的运算顺序。

3.5 数据类型转换

类型转换是将一个值从一种类型更改为另一种类型的过程。例如，可以将String类型数据“457”转换为一个数值型，而且可以将任意类型的数据转换为String类型。

如果从低精度数据类型向高精度数据类型转换，则永远不会溢出，并且总是成功的；而把高精度数据类型向低精度数据类型转换则必然会有信息丢失，有可能失败。

数据类型转换有两种方式，即隐式转换与显式转换。

3.5.1 隐式类型转换



视频讲解：光盘\TM\lx\3\隐式类型转换.exe

从低级类型向高级类型的转换，系统将自动执行，程序员无须进行任何操作。这种类型的转换称为隐式转换。下列基本数据类型会涉及数据转换，不包括逻辑类型和字符类型。这些类型按精度从低到高排列的顺序为byte < short < int < long < float < double。

【例3.20】使用int型变量为float型变量赋值，此时int型变量将隐式转换成float型变量。实例代码如下：

```
int x = 50;           //声明 int 型变量 x
float y = x;          //将 x 赋值给 y
```

此时执行输出语句，y的结果将是50.0。

隐式类型的转换也要遵循一定的规则，来解决在什么情况下将哪种类型的数据转换成另一种类型的数据。表3.10列出了各种数据类型转换的一般规则。

表3.10 隐式类型转换规则

操作数 1 的数据类型	操作数 2 的数据类型	转换后的数据类型
byte、short、char	int	int
byte、short、char、int、	long	long
byte、short、char、int、long	float	float
byte、short、char、int、long、float	double	double

下面通过一个简单实例介绍数据类型隐式转换。

【例3.21】在项目中创建类Conver，在主方法中创建不同数值型的变量，实现将各变量隐式转换。（实例位置：光盘\TM\sl\3.11）

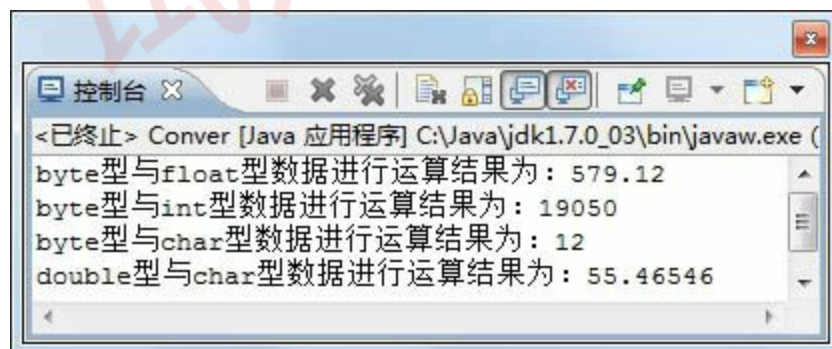
```

public class Conver {           //创建类
    public static void main(String[] args) {
        //定义 byte 型变量 mybyte, 并把 byte 型变量允许的最大值赋给 mybyte

        byte mybyte = 127;
        int myint = 150;         //定义 int 型变量 myint, 并赋值 150
        float myfloat = 452.12f; //定义 float 型变量 myfloat, 并赋值
        char mychar = 10;       //定义 char 型变量 mychar, 并赋值
        double mydouble = 45.46546; //定义 double 型变量, 并赋值
        System.out.println("byte 型与 float 型数据进行运算结果为: "
            + (mybyte + myfloat));
        /* 将运算结果输出 */
        System.out.println("byte 型与 int 型数据进行运算结果为: "
            + mybyte * myint);
        System.out.println("byte 型与 char 型数据进行运算结果为: "
            + mybyte / mychar);
        System.out.println("double 型与 char 型数据进行运算结果为: "
            + (mydouble + mychar));
    }
}

```

运行结果如图3.18所示。



```

<已终止> Conver [Java 应用程序] C:\Java\jdk1.7.0_03\bin\javaw.exe (
byte型与float型数据进行运算结果为: 579.12
byte型与int型数据进行运算结果为: 19050
byte型与char型数据进行运算结果为: 12
double型与char型数据进行运算结果为: 55.46546

```

图3.18 例3.21的运行结果



技巧

要理解类型转换，读者可以这么想象，大脑前面是一片内存，源和目标分别是两个大小不同的内存块（由变量及数据的类型来决定），将源数据赋值给目标内存的过程，就是用目标内存块尽可能多地套取源内存中的数据。

3.5.2 显式类型转换视频讲解：光盘\TM\lx\3\显式类型转换.exe

当把高精度的变量的值赋给低精度的变量时，必须使用显式类型转换运算（又称强制类型转换）。

语法如下：

(类型名)要转换的值

下面通过几种常见的显式数据类型转换实例来说明。

【例3.22】将不同的数据类型进行显式类型转换，实例代码如下：

```
int a = (int)45.23;           //此时输出 a 的值为 45
long y = (long)456.6F;        //此时输出 y 的值为 456
int b = (int)'d';             //此时输出 b 的值为 100
```

当执行显式类型转换时可能会导致精度损失。只要是boolean类型以外其他基本类型之间的转换，全部都能以显式类型转换的方法达到。



说明

当把整数赋值给一个byte、short、int、long型变量时，不可以超出这些变量的取值范围，否则必须进行强制类型转换。例如：

```
byte b = (byte)129;
```

3.6 代码注释与编码规范

在程序代码中适当地添加注释可以提高程序的可读性和可维护性。好的编码规范可以使程序更易阅读和理解。本节将介绍Java中的几种代码注释以及应该注意的编码规范。

3.6.1 代码注释



视频讲解：光盘\TM\lx\3\代码注释.exe

通过在程序代码中添加注释可提高程序的可读性。注释中包含了程序的信息，可以帮助程序员更好地阅读和理解程序。在Java源程序文件

的任意位置都可添加注释语句。注释中的文字Java编译器不进行编译，所有代码中的注释文字对程序不产生任何影响。Java语言提供了3种添加注释的方法，分别为单行注释、多行注释和文档注释。

1. 单行注释

“//”为单行注释标记，从符号“//”开始直到换行为止的所有内容均作为注释而被编译器忽略。

语法如下：

```
//注释内容
```

例如，以下代码为声明的int型变量添加注释：

```
int age;           //定义int型变量用于保存年龄信息
```

2. 多行注释

“/* */”为多行注释标记，符号“/*”与“*/”之间的所有内容均为注释内容。注释中的内容可以换行。

语法如下：

```
/*  
  注释内容 1  
  注释内容 2  
  ...  
*/
```



注意

在多行注释中可嵌套单行注释。例如：

```
/*  
    程序名称: Hello word //开发时间: 2008-03-05  
*/
```



注意

但在多行注释中不可以嵌套多行注释，以下代码为非法：


```
/*
  程序名称: Hello word
  /*开发时间: 2008-03-05
    作者: 张先生
*/
*/
```

3. 文档注释

“/** */”为文档注释标记。符号“/**”与“*/”之间的内容均为文档注释内容。当文档注释出现在声明（如类的声明、类的成员变量的声明、类的成员方法声明等）之前时，会被Javadoc文档工具读取作为Javadoc文档内容。文档注释的格式与多行注释的格式相同。对于初学者而言，文档注释并不是很重要，了解即可。



说明

一定要养成良好的编程风格。软件编码规范中提到“可读性第一，效率第二”，所以程序员必须要在程序中添加适量的注释来提高程序的可读性和可维护性。程序中注释要占程序代码总量的20%~50%。

3.6.2 编码规范



视频讲解：光盘\TM\lx\3\编码规范.exe

在学习开发的过程中要养成良好的编码习惯，因为规整的代码格式会给程序的开发与日后的维护提供很大方便。在此对编码规则作了以下总结，供读者学习。

- ☒ 每条语句要单独占一行，一条命令要以分号结束。



注意

程序代码中的分号必须为英文状态下输入的，初学者经常会将“;”写成中文状态下的“；”，此时编译器会报出illegal character（非法字符）这样的错误信息。

☑ 在声明变量时，尽量使每个变量的声明单独占一行，即使是相同的数据类型也要将其放置在单独的一行上，这样有助于添加注释。对于局部变量应在声明的同时对其进行初始化。

☑ 在Java代码中，关键字与关键字间如果有多个空格，这些空格均被视作一个。例如：多行空格没有任何意义，为了便于理解、阅读，应控制好空格的数量。

等价于

```
public static void main(String args[]) ⇨ public static void main(String args[])
```

☑ 为了方便日后的维护，不要使用技术性很高、难懂、易混淆判断的语句。由于程序的开发与维护不能是同一个人，所以应尽量使用简单的技术完成程序需要的功能。

☑ 对于关键的方法要多加注释，这样有助于阅读者了解代码结构。

3.7 小 结

本章向读者介绍的是Java语言基础，其中需要读者重点掌握的是Java语言的基本数据类型、变量与常量以及运算符三大知识点。初学者经常会将String类型认为是Java语言的基本数据类型，在此提醒读者Java语言的基本数据类型中并没有String类型。另外，要对数据类型之间的转换有一定的了解。在使用变量时，需要读者注意的是变量的有效范围，否则在使用时会出现编译错误或浪费内存资源。此外，Java中的各种运算符也是Java基础中的重点，正确使用这些运算符，才能得到预期的结果。

3.8 实践与练习

1. 编写Java程序将两个数相加的结果输出。（答案位置：光盘\TM\sl\3.12）

2. 编写Java程序，声明成员变量age与局部变量name。比较一下两个变量的区别，并添加相应的注释。（答案位置：光盘\TM\sl\3.13）

3. 编写Java程序，将所有整型数值全部转换成int型，并将转换后的值输出。（答案位置：光盘\TM\sl\3.14）